

ORACLE
PRESS



CORE JAVA

Volume I: Fundamentals

TWELFTH EDITION

ORACLE

Cay S. Horstmann

Core Java



Volume I: Fundamentals

Twelfth Edition

This page intentionally left blank

Core Java

Volume I: Fundamentals

Twelfth Edition

Cay S. Horstmann



Pearson

Boston • Columbus • New York • San Francisco • Amsterdam • Cape Town

Dubai • London • Madrid • Milan • Munich • Paris • Montreal • Toronto • Delhi • Mexico City

São Paulo • Sydney • Hong Kong • Seoul • Singapore • Taipei • Tokyo

Cover image: Jon Chica/Shutterstock

Figure 1.1: © Jmol

Figures 2.1-2.3, 2.9: © Microsoft 2021

Figures 2.5-2.8: © Eclipse Foundation

Figures 3.2-3.5, 4.9, 3.2-3.5, 4.9, 5.4, 7.2, 7.3, 10.1, 10.3, 10.8, 10.10, 10.11, 10.14-10.16, 11.4, 11.5, 11.8-11.34, 12.5, 12.6: © 2021 Oracle

The author and publisher have taken care in the preparation of this book, but make no expressed or implied warranty of any kind and assume no responsibility for errors or omissions. No liability is assumed for incidental or consequential damages in connection with or arising out of the use of the information or programs contained herein.

The views expressed in this book are those of the author and do not necessarily reflect the views of Oracle.

For information about buying this title in bulk quantities, or for special sales opportunities (which may include electronic versions; custom cover designs; and content particular to your business, training goals, marketing focus, or branding interests), please contact our corporate sales department at corpsales@pearsoned.com or (800) 382-3419.

For government sales inquiries, please contact governmentsales@pearsoned.com.

For questions about sales outside the United States, please contact international@pearsoned.com.

Visit us on the Web: informit.com

Library of Congress Preassigned Control Number: 2021946079

Copyright © 2022 Pearson Education Inc.

Portions copyright © 1996-2013 Oracle and/or its affiliates. All Rights Reserved.

Oracle America Inc. does not make any representations or warranties as to the accuracy, adequacy or completeness of any information contained in this work, and is not responsible for any errors or omissions.

Microsoft and/or its respective suppliers make no representations about the suitability of the information contained in the documents and related graphics published as part of the services for any purpose. All such documents and related graphics are provided "as is" without warranty of any kind. Microsoft and/or its respective suppliers hereby disclaim all warranties and conditions with regard to this information, including all warranties and conditions of merchantability, whether express, implied or statutory, fitness for a particular purpose, title and non-infringement. In no event shall Microsoft and/or its respective suppliers be liable for any special, indirect or consequential damages or any damages whatsoever resulting from loss of use, data or profits, whether in an action of contract, negligence or other tortious action, arising out of or in connection with the use or performance of information available from the services. The documents and related graphics contained herein could include technical inaccuracies or typographical errors. Changes are periodically added to the information herein. Microsoft and/or its respective suppliers may make improvements and/or changes in the product(s) and/or the program(s) described herein at any time. Partial screen shots may be viewed in full within the software version specified.

Microsoft® Windows®, and Microsoft Office® are registered trademarks of the Microsoft Corporation in the U.S.A. and other countries. This book is not sponsored or endorsed by or affiliated with the Microsoft Corporation.

All rights reserved. This publication is protected by copyright, and permission must be obtained from the publisher prior to any prohibited reproduction, storage in a retrieval system, or transmission in any form or by any means, electronic, mechanical, photocopying, recording, or likewise. For information regarding permissions, request forms and the appropriate contacts within the Pearson Education Global Rights & Permissions Department, please visit www.pearson.com/permissions/.

ISBN-13: 978-0-13-767362-9

ISBN-10: 0-13-767362-0

ScoutAutomatedPrintCode



Humans are born to learn

Learning is more than just gaining knowledge. It's a vital force that impacts all aspects of our lives.

As the world's lifelong learning company, we help people realize the life they imagine through learning. That's why our products and services are designed by experts and backed by learning science. We deliver trusted, quality content and resources from the authors, creators, innovators, and leaders of business and technology, via the most storied imprints in IT, including Addison-Wesley, Cisco Press, Adobe Press, Microsoft Press, and Pearson IT Certification.

Our team draws inspiration from IT communities around the globe to publish on the topics that matter most to application and software developers and designers, programmers, system administrators, hobbyists, and IT professionals.

We are a community powered by passionate authors and teams—and you. Your support helps us to continue to offer resources that meet the needs of today's professionals.

**Because the more we learn,
the more we live.**

Learn more about Pearson at [Pearson.com](https://www.pearson.com)



This page intentionally left blank

Contents

<i>Preface</i>	<i>xxi</i>
<i>Acknowledgments</i>	<i>xxvii</i>
Chapter 1: An Introduction to Java	1
1.1 Java as a Programming Platform	1
1.2 The Java “White Paper” Buzzwords	2
1.2.1 Simple	3
1.2.2 Object-Oriented	4
1.2.3 Distributed	4
1.2.4 Robust	4
1.2.5 Secure	5
1.2.6 Architecture-Neutral	6
1.2.7 Portable	6
1.2.8 Interpreted	7
1.2.9 High-Performance	7
1.2.10 Multithreaded	8
1.2.11 Dynamic	8
1.3 Java Applets and the Internet	9
1.4 A Short History of Java	10
1.5 Common Misconceptions about Java	13
Chapter 2: The Java Programming Environment	17
2.1 Installing the Java Development Kit	18
2.1.1 Downloading the JDK	18
2.1.2 Setting Up the JDK	18
2.1.3 Installing Source Files and Documentation	20
2.2 Using the Command-Line Tools	22
2.3 Using an Integrated Development Environment	27
2.4 JShell	30

Chapter 3: Fundamental Programming Structures in Java	35
3.1 A Simple Java Program	36
3.2 Comments	39
3.3 Data Types	40
3.3.1 Integer Types	40
3.3.2 Floating-Point Types	42
3.3.3 The char Type	43
3.3.4 Unicode and the char Type	45
3.3.5 The boolean Type	46
3.4 Variables and Constants	46
3.4.1 Declaring Variables	47
3.4.2 Initializing Variables	48
3.4.3 Constants	49
3.4.4 Enumerated Types	50
3.5 Operators	51
3.5.1 Arithmetic Operators	51
3.5.2 Mathematical Functions and Constants	51
3.5.3 Conversions between Numeric Types	53
3.5.4 Casts	54
3.5.5 Assignment	55
3.5.6 Increment and Decrement Operators	56
3.5.7 Relational and boolean Operators	57
3.5.8 The Conditional Operator	58
3.5.9 Switch Expressions	58
3.5.10 Bitwise Operators	59
3.5.11 Parentheses and Operator Hierarchy	60
3.6 Strings	61
3.6.1 Substrings	62
3.6.2 Concatenation	62
3.6.3 Strings Are Immutable	63
3.6.4 Testing Strings for Equality	64
3.6.5 Empty and Null Strings	65
3.6.6 Code Points and Code Units	66
3.6.7 The String API	68
3.6.8 Reading the Online API Documentation	70

3.6.9	Building Strings	73
3.6.10	Text Blocks	74
3.7	Input and Output	76
3.7.1	Reading Input	76
3.7.2	Formatting Output	79
3.7.3	File Input and Output	82
3.8	Control Flow	85
3.8.1	Block Scope	85
3.8.2	Conditional Statements	86
3.8.3	Loops	89
3.8.4	Determinate Loops	94
3.8.5	Multiple Selections with <code>switch</code>	98
3.8.6	Statements That Break Control Flow	103
3.9	Big Numbers	106
3.10	Arrays	109
3.10.1	Declaring Arrays	109
3.10.2	Accessing Array Elements	111
3.10.3	The “for each” Loop	112
3.10.4	Array Copying	113
3.10.5	Command-Line Parameters	114
3.10.6	Array Sorting	115
3.10.7	Multidimensional Arrays	118
3.10.8	Ragged Arrays	121
Chapter 4: Objects and Classes		125
4.1	Introduction to Object-Oriented Programming	126
4.1.1	Classes	127
4.1.2	Objects	128
4.1.3	Identifying Classes	129
4.1.4	Relationships between Classes	130
4.2	Using Predefined Classes	132
4.2.1	Objects and Object Variables	132
4.2.2	The <code>LocalDate</code> Class of the Java Library	136
4.2.3	Mutator and Accessor Methods	138
4.3	Defining Your Own Classes	141
4.3.1	An <code>Employee</code> Class	142

4.3.2	Use of Multiple Source Files	145
4.3.3	Dissecting the <code>Employee</code> Class	146
4.3.4	First Steps with Constructors	146
4.3.5	Declaring Local Variables with <code>var</code>	148
4.3.6	Working with <code>null</code> References	149
4.3.7	Implicit and Explicit Parameters	150
4.3.8	Benefits of Encapsulation	151
4.3.9	Class-Based Access Privileges	154
4.3.10	Private Methods	155
4.3.11	Final Instance Fields	155
4.4	Static Fields and Methods	156
4.4.1	Static Fields	156
4.4.2	Static Constants	157
4.4.3	Static Methods	158
4.4.4	Factory Methods	159
4.4.5	The <code>main</code> Method	160
4.5	Method Parameters	163
4.6	Object Construction	170
4.6.1	Overloading	170
4.6.2	Default Field Initialization	171
4.6.3	The Constructor with No Arguments	172
4.6.4	Explicit Field Initialization	173
4.6.5	Parameter Names	174
4.6.6	Calling Another Constructor	175
4.6.7	Initialization Blocks	175
4.6.8	Object Destruction and the <code>finalize</code> Method	180
4.7	Records	181
4.7.1	The Record Concept	182
4.7.2	Constructors: Canonical, Custom, and Compact	184
4.8	Packages	186
4.8.1	Package Names	186
4.8.2	Class Importation	187
4.8.3	Static Imports	189
4.8.4	Addition of a Class into a Package	190
4.8.5	Package Access	193

4.8.6	The Class Path	195
4.8.7	Setting the Class Path	197
4.9	JAR Files	198
4.9.1	Creating JAR files	198
4.9.2	The Manifest	199
4.9.3	Executable JAR Files	200
4.9.4	Multi-Release JAR Files	201
4.9.5	A Note about Command-Line Options	202
4.10	Documentation Comments	204
4.10.1	Comment Insertion	204
4.10.2	Class Comments	205
4.10.3	Method Comments	206
4.10.4	Field Comments	207
4.10.5	General Comments	207
4.10.6	Package Comments	208
4.10.7	Comment Extraction	209
4.11	Class Design Hints	210
Chapter 5:	Inheritance	213
5.1	Classes, Superclasses, and Subclasses	214
5.1.1	Defining Subclasses	214
5.1.2	Overriding Methods	216
5.1.3	Subclass Constructors	218
5.1.4	Inheritance Hierarchies	222
5.1.5	Polymorphism	223
5.1.6	Understanding Method Calls	225
5.1.7	Preventing Inheritance: Final Classes and Methods	228
5.1.8	Casting	229
5.1.9	Pattern Matching for instanceof	232
5.1.10	Protected Access	234
5.2	Object: The Cosmic Superclass	235
5.2.1	Variables of Type Object	235
5.2.2	The equals Method	236
5.2.3	Equality Testing and Inheritance	238
5.2.4	The hashCode Method	241
5.2.5	The toString Method	244

5.3	Generic Array Lists	251
5.3.1	Declaring Array Lists	252
5.3.2	Accessing Array List Elements	254
5.3.3	Compatibility between Typed and Raw Array Lists	258
5.4	Object Wrappers and Autoboxing	259
5.5	Methods with a Variable Number of Parameters	263
5.6	Abstract Classes	265
5.7	Enumeration Classes	271
5.8	Sealed Classes	273
5.9	Reflection	279
5.9.1	The Class Class	280
5.9.2	A Primer on Declaring Exceptions	283
5.9.3	Resources	284
5.9.4	Using Reflection to Analyze the Capabilities of Classes .	287
5.9.5	Using Reflection to Analyze Objects at Runtime	294
5.9.6	Using Reflection to Write Generic Array Code	300
5.9.7	Invoking Arbitrary Methods and Constructors	304
5.10	Design Hints for Inheritance	308
Chapter 6: Interfaces, Lambda Expressions, and Inner Classes		311
6.1	Interfaces	312
6.1.1	The Interface Concept	312
6.1.2	Properties of Interfaces	319
6.1.3	Interfaces and Abstract Classes	321
6.1.4	Static and Private Methods	322
6.1.5	Default Methods	323
6.1.6	Resolving Default Method Conflicts	324
6.1.7	Interfaces and Callbacks	326
6.1.8	The Comparator Interface	329
6.1.9	Object Cloning	330
6.2	Lambda Expressions	338
6.2.1	Why Lambdas?	338
6.2.2	The Syntax of Lambda Expressions	339
6.2.3	Functional Interfaces	342
6.2.4	Method References	344
6.2.5	Constructor References	348

6.2.6	Variable Scope	349
6.2.7	Processing Lambda Expressions	352
6.2.8	More about Comparators	356
6.3	Inner Classes	357
6.3.1	Use of an Inner Class to Access Object State	358
6.3.2	Special Syntax Rules for Inner Classes	362
6.3.3	Are Inner Classes Useful? Actually Necessary? Secure?	363
6.3.4	Local Inner Classes	365
6.3.5	Accessing Variables from Outer Methods	366
6.3.6	Anonymous Inner Classes	367
6.3.7	Static Inner Classes	372
6.4	Service Loaders	376
6.5	Proxies	378
6.5.1	When to Use Proxies	379
6.5.2	Creating Proxy Objects	379
6.5.3	Properties of Proxy Classes	383
Chapter 7: Exceptions, Assertions, and Logging		387
7.1	Dealing with Errors	388
7.1.1	The Classification of Exceptions	389
7.1.2	Declaring Checked Exceptions	391
7.1.3	How to Throw an Exception	394
7.1.4	Creating Exception Classes	396
7.2	Catching Exceptions	397
7.2.1	Catching an Exception	397
7.2.2	Catching Multiple Exceptions	399
7.2.3	Rethrowing and Chaining Exceptions	400
7.2.4	The finally Clause	402
7.2.5	The try-with-Resources Statement	405
7.2.6	Analyzing Stack Trace Elements	407
7.3	Tips for Using Exceptions	411
7.4	Using Assertions	415
7.4.1	The Assertion Concept	415
7.4.2	Assertion Enabling and Disabling	416
7.4.3	Using Assertions for Parameter Checking	417
7.4.4	Using Assertions for Documenting Assumptions	419

7.5	Logging	420
7.5.1	Basic Logging	421
7.5.2	Advanced Logging	421
7.5.3	Changing the Log Manager Configuration	424
7.5.4	Localization	426
7.5.5	Handlers	427
7.5.6	Filters	431
7.5.7	Formatters	431
7.5.8	A Logging Recipe	432
7.6	Debugging Tips	441
Chapter 8: Generic Programming		447
8.1	Why Generic Programming?	448
8.1.1	The Advantage of Type Parameters	448
8.1.2	Who Wants to Be a Generic Programmer?	449
8.2	Defining a Simple Generic Class	450
8.3	Generic Methods	453
8.4	Bounds for Type Variables	454
8.5	Generic Code and the Virtual Machine	457
8.5.1	Type Erasure	457
8.5.2	Translating Generic Expressions	458
8.5.3	Translating Generic Methods	459
8.5.4	Calling Legacy Code	461
8.6	Restrictions and Limitations	462
8.6.1	Type Parameters Cannot Be Instantiated with Primitive Types	463
8.6.2	Runtime Type Inquiry Only Works with Raw Types	463
8.6.3	You Cannot Create Arrays of Parameterized Types	463
8.6.4	Varargs Warnings	464
8.6.5	You Cannot Instantiate Type Variables	465
8.6.6	You Cannot Construct a Generic Array	466
8.6.7	Type Variables Are Not Valid in Static Contexts of Generic Classes	468
8.6.8	You Cannot Throw or Catch Instances of a Generic Class	469
8.6.9	You Can Defeat Checked Exception Checking	469

8.6.10	Beware of Clashes after Erasure	471
8.7	Inheritance Rules for Generic Types	472
8.8	Wildcard Types	475
8.8.1	The Wildcard Concept	475
8.8.2	Supertype Bounds for Wildcards	476
8.8.3	Unbounded Wildcards	480
8.8.4	Wildcard Capture	480
8.9	Reflection and Generics	483
8.9.1	The Generic Class Class	483
8.9.2	Using <code>Class<T></code> Parameters for Type Matching	484
8.9.3	Generic Type Information in the Virtual Machine	485
8.9.4	Type Literals	489
Chapter 9:	Collections	497
9.1	The Java Collections Framework	498
9.1.1	Separating Collection Interfaces and Implementation	498
9.1.2	The Collection Interface	501
9.1.3	Iterators	501
9.1.4	Generic Utility Methods	504
9.2	Interfaces in the Collections Framework	508
9.3	Concrete Collections	510
9.3.1	Linked Lists	512
9.3.2	Array Lists	523
9.3.3	Hash Sets	523
9.3.4	Tree Sets	527
9.3.5	Queues and Deques	532
9.3.6	Priority Queues	533
9.4	Maps	535
9.4.1	Basic Map Operations	535
9.4.2	Updating Map Entries	539
9.4.3	Map Views	540
9.4.4	Weak Hash Maps	542
9.4.5	Linked Hash Sets and Maps	543
9.4.6	Enumeration Sets and Maps	545
9.4.7	Identity Hash Maps	545
9.5	Copies and Views	548

9.5.1	Small Collections	548
9.5.2	Unmodifiable Copies and Views	550
9.5.3	Subranges	552
9.5.4	Checked Views	553
9.5.5	Synchronized Views	553
9.5.6	A Note on Optional Operations	554
9.6	Algorithms	558
9.6.1	Why Generic Algorithms?	558
9.6.2	Sorting and Shuffling	560
9.6.3	Binary Search	563
9.6.4	Simple Algorithms	564
9.6.5	Bulk Operations	566
9.6.6	Converting between Collections and Arrays	567
9.6.7	Writing Your Own Algorithms	568
9.7	Legacy Collections	569
9.7.1	The Hashtable Class	570
9.7.2	Enumerations	570
9.7.3	Property Maps	572
9.7.4	Stacks	575
9.7.5	Bit Sets	576
Chapter 10: Graphical User Interface Programming		581
10.1	A History of Java User Interface Toolkits	582
10.2	Displaying Frames	583
10.2.1	Creating a Frame	584
10.2.2	Frame Properties	586
10.3	Displaying Information in a Component	590
10.3.1	Working with 2D Shapes	595
10.3.2	Using Color	603
10.3.3	Using Fonts	605
10.3.4	Displaying Images	612
10.4	Event Handling	614
10.4.1	Basic Event Handling Concepts	614
10.4.2	Example: Handling a Button Click	616
10.4.3	Specifying Listeners Concisely	620
10.4.4	Adapter Classes	621

10.4.5	Actions	623
10.4.6	Mouse Events	629
10.4.7	The AWT Event Hierarchy	636
10.5	The Preferences API	639
Chapter 11: User Interface Components with Swing		647
11.1	Swing and the Model-View-Controller Design Pattern	648
11.2	Introduction to Layout Management	652
11.2.1	Layout Managers	652
11.2.2	Border Layout	655
11.2.3	Grid Layout	657
11.3	Text Input	658
11.3.1	Text Fields	659
11.3.2	Labels and Labeling Components	661
11.3.3	Password Fields	662
11.3.4	Text Areas	663
11.3.5	Scroll Panes	664
11.4	Choice Components	667
11.4.1	Checkboxes	667
11.4.2	Radio Buttons	670
11.4.3	Borders	673
11.4.4	Combo Boxes	676
11.4.5	Sliders	680
11.5	Menus	686
11.5.1	Menu Building	687
11.5.2	Icons in Menu Items	690
11.5.3	Checkbox and Radio Button Menu Items	691
11.5.4	Pop-Up Menus	692
11.5.5	Keyboard Mnemonics and Accelerators	694
11.5.6	Enabling and Disabling Menu Items	696
11.5.7	Toolbars	701
11.5.8	Tooltips	704
11.6	Sophisticated Layout Management	705
11.6.1	The Grid Bag Layout	706
11.6.1.1	The gridx, gridy, gridwidth, and gridheight Parameters	708

11.6.1.2	Weight Fields	708
11.6.1.3	The fill and anchor Parameters	709
11.6.1.4	Padding	709
11.6.1.5	Alternative Method to Specify the gridx, gridy, gridwidth, and gridheight Parameters	709
11.6.1.6	A Grid Bag Layout Recipe	710
11.6.1.7	A Helper Class to Tame the Grid Bag Constraints	710
11.6.2	Custom Layout Managers	716
11.7	Dialog Boxes	721
11.7.1	Option Dialogs	722
11.7.2	Creating Dialogs	726
11.7.3	Data Exchange	730
11.7.4	File Dialogs	737
Chapter 12	Concurrency	747
12.1	What Are Threads?	748
12.2	Thread States	753
12.2.1	New Threads	754
12.2.2	Runnable Threads	754
12.2.3	Blocked and Waiting Threads	755
12.2.4	Terminated Threads	756
12.3	Thread Properties	757
12.3.1	Interrupting Threads	757
12.3.2	Daemon Threads	761
12.3.3	Thread Names	761
12.3.4	Handlers for Uncaught Exceptions	761
12.3.5	Thread Priorities	763
12.4	Synchronization	764
12.4.1	An Example of a Race Condition	764
12.4.2	The Race Condition Explained	766
12.4.3	Lock Objects	769
12.4.4	Condition Objects	772
12.4.5	The synchronized Keyword	778
12.4.6	Synchronized Blocks	782
12.4.7	The Monitor Concept	784

12.4.8	Volatile Fields	785
12.4.9	Final Variables	787
12.4.10	Atomics	787
12.4.11	Deadlocks	789
12.4.12	Why the stop and suspend Methods Are Deprecated	793
12.4.13	On-Demand Initialization	794
12.4.14	Thread-Local Variables	795
12.5	Thread-Safe Collections	797
12.5.1	Blocking Queues	797
12.5.2	Efficient Maps, Sets, and Queues	805
12.5.3	Atomic Update of Map Entries	806
12.5.4	Bulk Operations on Concurrent Hash Maps	810
12.5.5	Concurrent Set Views	812
12.5.6	Copy on Write Arrays	813
12.5.7	Parallel Array Algorithms	813
12.5.8	Older Thread-Safe Collections	814
12.6	Tasks and Thread Pools	815
12.6.1	Callables and Futures	816
12.6.2	Executors	818
12.6.3	Controlling Groups of Tasks	821
12.6.4	The Fork-Join Framework	827
12.7	Asynchronous Computations	830
12.7.1	Completable Futures	830
12.7.2	Composing Completable Futures	832
12.7.3	Long-Running Tasks in User Interface Callbacks	839
12.8	Processes	847
12.8.1	Building a Process	847
12.8.2	Running a Process	849
12.8.3	Process Handles	850
Appendix		855
<i>Index</i>		<i>861</i>

This page intentionally left blank

Preface

To the Reader

In late 1995, the Java programming language burst onto the Internet scene and gained instant celebrity status. The promise of Java technology was that it would become the *universal glue* that connects users with information wherever it comes from—web servers, databases, information providers, or any other imaginable source. Indeed, Java is in a unique position to fulfill this promise. It is an extremely solidly engineered language that has gained wide acceptance. Its built-in security and safety features are reassuring both to programmers and to the users of Java programs. Java has built-in support for advanced programming tasks, such as network programming, database connectivity, and concurrency.

Since 1995, twelve major revisions of the Java Development Kit have been released. Over the course of the last 25 years, the Application Programming Interface (API) has grown from about 200 to over 4,000 classes. The API now spans such diverse areas as user interface construction, database management, internationalization, security, and XML processing.

The book that you are reading right now is the first volume of the twelfth edition of *Core Java*. Each edition closely followed a release of the Java Development Kit, and each time, I rewrote the book to take advantage of the newest Java features. This edition has been updated to reflect the features of Java 17.

As with the previous editions, *this book still targets serious programmers who want to put Java to work on real projects*. I think of you, the reader, as a programmer with a solid background in a programming language other than Java. I assume that you don't like books filled with toy examples (such as toasters, zoo animals, or "nervous text"). You won't find any of these in the book. My goal is to enable you to fully understand the Java language and library, not to give you an illusion of understanding.

In this book you will find lots of sample code demonstrating almost every language and library feature. The sample programs are purposefully simple to focus on the major points, but, for the most part, they aren't fake and they don't cut corners. They should make good starting points for your own code.

I assume you are willing, even eager, to learn about all the advanced features that Java puts at your disposal. For example, you will find a detailed treatment of

- Object-oriented programming
- Reflection and proxies
- Interfaces and inner classes
- Exception handling
- Generic programming
- The collections framework
- The event listener model
- Graphical user interface design
- Concurrency

With the explosive growth of the Java class library, a one-volume treatment of all the features of Java that serious programmers need to know is no longer possible. Hence, the book is broken up into two volumes. This first volume concentrates on the fundamental concepts of the Java language, along with the basics of user-interface programming. The second volume, *Core Java, Volume II: Advanced Features*, goes further into the enterprise features and advanced user-interface programming. It includes detailed discussions of

- The Stream API
- File processing and regular expressions
- Databases
- XML processing
- Annotations
- Internationalization
- Network programming
- Advanced GUI components
- Advanced graphics
- Native methods

When writing a book, errors and inaccuracies are inevitable. I'd very much like to know about them. But, of course, I'd prefer to learn about each of them only once. You will find a list of frequently asked questions and bug fixes at <http://horstmann.com/corejava>. Strategically placed at the end of the errata page (to encourage you to read through it first) is a form you can use to report bugs and suggest improvements. Please don't be disappointed if I don't answer

every query or don't get back to you immediately. I do read all e-mail and appreciate your input to make future editions of this book clearer and more informative.

A Tour of This Book

Chapter 1 gives an overview of the capabilities of Java that set it apart from other programming languages. The chapter explains what the designers of the language set out to do and to what extent they succeeded. A short history of Java follows, detailing how Java came into being and how it has evolved.

In **Chapter 2**, you will see how to download and install the JDK and the program examples for this book. Then I'll guide you through compiling and running a console application and a graphical application. You will see how to use the plain JDK, a Java IDE, and the JShell tool.

Chapter 3 starts the discussion of the Java language. In this chapter, I cover the basics: variables, loops, and simple functions. If you are a C or C++ programmer, this is smooth sailing because the syntax for these language features is essentially the same as in C. If you come from a non-C background such as Visual Basic, you will want to read this chapter carefully.

Object-oriented programming (OOP) is now in the mainstream of programming practice, and Java is an object-oriented programming language. **Chapter 4** introduces encapsulation, the first of two fundamental building blocks of object orientation, and the Java language mechanism to implement it—that is, classes and methods. In addition to the rules of the Java language, you will also find advice on sound OOP design. Finally, I cover the marvelous javadoc tool that formats your code comments as a set of hyperlinked web pages. If you are familiar with C++, you can browse through this chapter quickly. Programmers coming from a non-object-oriented background should expect to spend some time mastering the OOP concepts before going further with Java.

Classes and encapsulation are only one part of the OOP story, and **Chapter 5** introduces the other—namely, *inheritance*. Inheritance lets you take an existing class and modify it according to your needs. This is a fundamental technique for programming in Java. The inheritance mechanism in Java is quite similar to that in C++. Once again, C++ programmers can focus on the differences between the languages.

Chapter 6 shows you how to use Java's notion of an *interface*. Interfaces let you go beyond the simple inheritance model of Chapter 5. Mastering interfaces

allows you to have full access to the power of Java's completely object-oriented approach to programming. After covering interfaces, I move on to *lambda expressions*, a concise way for expressing a block of code that can be executed at a later point in time. I then explain a useful technical feature of Java called *inner classes*.

Chapter 7 discusses *exception handling*—Java's robust mechanism to deal with the fact that bad things can happen to good programs. Exceptions give you an efficient way of separating the normal processing code from the error handling. Of course, even after hardening your program by handling all exceptional conditions, it still might fail to work as expected. In the final part of this chapter, I give you a number of useful debugging tips.

Chapter 8 gives an overview of generic programming. Generic programming makes your programs easier to read and safer. I show you how to use strong typing and remove unsightly and unsafe casts, and how to deal with the complexities that arise from the need to stay compatible with older versions of Java.

The topic of **Chapter 9** is the collections framework of the Java platform. Whenever you want to collect multiple objects and retrieve them later, you should use a collection that is best suited for your circumstances, instead of just tossing the elements into an array. This chapter shows you how to take advantage of the standard collections that are prebuilt for your use.

Chapter 10 provides an introduction into GUI programming. I show how you can make windows, how to paint on them, how to draw with geometric shapes, how to format text in multiple fonts, and how to display images. Next, you'll see how to write code that responds to events, such as mouse clicks or key presses.

Chapter 11 discusses the Swing GUI toolkit in great detail. The Swing toolkit allows you to build cross-platform graphical user interfaces. You'll learn all about the various kinds of buttons, text components, borders, sliders, list boxes, menus, and dialog boxes. However, some of the more advanced components are discussed in Volume II.

Chapter 12 finishes the book with a discussion of concurrency, which enables you to program tasks to be done in parallel. This is an important and exciting application of Java technology in an era where most processors have multiple cores that you want to keep busy.

The **Appendix** lists the reserved words of the Java language.

Conventions

As is common in many computer books, I use monospace type to represent computer code.



NOTE: Notes are tagged with “note” icons that look like this.



TIP: Tips are tagged with “tip” icons that look like this.



CAUTION: When there is danger ahead, I warn you with a “caution” icon.



C++ NOTE: There are many C++ notes that explain the differences between Java and C++. You can skip over them if you don’t have a background in C++ or if you consider your experience with that language a bad dream of which you’d rather not be reminded.

Java comes with a large programming library, or Application Programming Interface (API). When using an API call for the first time, I add a short summary description at the end of the section. These descriptions are a bit more informal but, hopefully, also a little more informative than those in the official online API documentation. The names of interfaces are in italics, just like in the official documentation. The number after a class, interface, or method name is the JDK version in which the feature was introduced, as shown in the following example:

Application Programming Interface 9

Programs whose source code is on the book’s companion web site are presented as listings, for instance:

Listing 1.1 InputTest/InputTest.java

Sample Code

The web site for this book at <http://horstmann.com/corejava> contains all sample code from the book. See Chapter 2 for more information on installing the Java Development Kit and the sample code.

Register your copy of *Core Java, Volume I: Fundamentals, Twelfth Edition*, on the InformIT site for convenient access to updates and/or corrections as they become available. To start the registration process, go to informit.com/register and log in or create an account. Enter the product ISBN (9780137673629) and click Submit. Look on the Registered Products tab for an Access Bonus Content link next to this product, and follow that link to access any available bonus materials. If you would like to be notified of exclusive offers on new editions and updates, please check the box to receive email from us.

Acknowledgments

Writing a book is always a monumental effort, and rewriting it doesn't seem to be much easier, especially with the continuous change in Java technology. Making a book a reality takes many dedicated people, and it is my great pleasure to acknowledge the contributions of the entire *Core Java* team.

A large number of individuals at Pearson provided valuable assistance but managed to stay behind the scenes. I'd like them all to know how much I appreciate their efforts. As always, my warm thanks go to my editor, Greg Doench, for steering the book through the writing and production process, and for allowing me to be blissfully unaware of the existence of all those folks behind the scenes. I am very grateful to Julie Nahil for production support, and to Dmitry Kirsanov and Alina Kirsanova for copyediting and typesetting the manuscript. My thanks also to my coauthor of earlier editions, Gary Cornell, who has since moved on to other ventures.

Thanks to the many readers of earlier editions who reported errors and made lots of thoughtful suggestions for improvement. I am particularly grateful to the excellent reviewing team who went over the manuscript with an amazing eye for detail and saved me from many embarrassing errors.

Reviewers of this and earlier editions include Chuck Allison (Utah Valley University), Lance Andersen (Oracle), Gail Anderson (Anderson Software Group), Paul Anderson (Anderson Software Group), Alec Beaton (IBM), Cliff Berg, Andrew Binstock (Oracle), Joshua Bloch, David Brown, Corky Cartwright, Frank Cohen (PushToTest), Chris Crane (devXsolution), Dr. Nicholas J. De Lillo (Manhattan College), Rakesh Dhoopar (Oracle), David Geary (Clarity Training), Jim Gish (Oracle), Brian Goetz (Oracle), Angela Gordon, Dan Gordon (Electric Cloud), Rob Gordon, John Gray (University of Hartford), Cameron Gregory (olabs.com), Marty Hall (coreservlets.com, Inc.), Vincent Hardy (Adobe Systems), Dan Harkey (San Jose State University), William Higgins (IBM), Marc Hoffmann (mtrail), Vladimir Ivanovic (PointBase), Jerry Jackson (CA Technologies), Heinz Kabutz (Java Specialists), Stepan V. Kalinin (I-Teco/Servionica LTD), Tim Kimmet (Walmart), Chris Laffra, Charlie Lai (Apple), Angelika Langer, Jeff Langr (Langr Software Solutions), Doug Langston, Hang Lau (McGill University), Mark Lawrence, Doug Lea (SUNY Oswego), Gregory Longshore, Bob Lynch (Lynch Associates), Philip Milne (consultant), Mark Morrissey (The Oregon Graduate Institute), Mahesh Nee-lakanta (Florida Atlantic University), José Paumard (Oracle), Hao Pham, Paul

Philion, Blake Ragsdell, Stuart Reges (University of Arizona), Simon Ritter (Azul Systems), Rich Rosen (Interactive Data Corporation), Peter Sanders (ESSI University, Nice, France), Dr. Paul Sanghera (San Jose State University and Brooks College), Paul Sevinc (Teamup AG), Devang Shah (Sun Microsystems), Yoshiki Shibata, Bradley A. Smith, Steven Stelting (Oracle), Christopher Taylor, Luke Taylor (Valtech), George Thiruvathukal, Kim Topley (StreamingEdge), Janet Traub, Paul Tyma (consultant), Peter van der Linden, Christian Ullenboom, Burt Walsh, Dan Xu (Oracle), and John Zavgren (Oracle).

Cay Horstmann
Berlin, Germany
October 2021

An Introduction to Java

In this chapter

- 1.1 Java as a Programming Platform, page 1
- 1.2 The Java “White Paper” Buzzwords, page 2
- 1.3 Java Applets and the Internet, page 9
- 1.4 A Short History of Java, page 10
- 1.5 Common Misconceptions about Java, page 13

The first release of Java in 1996 generated an incredible amount of excitement, not just in the computer press, but in mainstream media such as *The New York Times*, *The Washington Post*, and *BusinessWeek*. Java has the distinction of being the first and only programming language that had a ten-minute story on National Public Radio. A \$100,000,000 venture capital fund was set up solely for products using a *specific* computer language. I hope you will enjoy a brief history of Java that you will find in this chapter.

1.1 Java as a Programming Platform

In the first edition of this book, my coauthor Gary Cornell and I had this to write about Java:

“As a computer language, Java’s hype is overdone: Java is certainly a *good* programming language. There is no doubt that it is one of the better languages available to serious programmers. We think it could *potentially* have been a

great programming language, but it is probably too late for that. Once a language is out in the field, the ugly reality of compatibility with existing code sets in.”

Our editor got a lot of flack for this paragraph from someone very high up at Sun Microsystems, the company that originally developed Java. The Java language has a lot of nice features that we will examine in detail later in this chapter. It has its share of warts, and some of the newer additions to the language are not as elegant as the original features because of compatibility requirements.

But, as we already said in the first edition, Java was never just a language. There are lots of programming languages out there, but few of them make much of a splash. Java is a whole *platform*, with a huge library, containing lots of reusable code, and an execution environment that provides services such as security, portability across operating systems, and automatic garbage collection.

As a programmer, you will want a language with a pleasant syntax and comprehensible semantics (i.e., not C++). Java fits the bill, as do dozens of other fine languages. Some languages give you portability, garbage collection, and the like, but they don’t have much of a library, forcing you to roll your own if you want fancy graphics or networking or database access. Well, Java has everything—a good language, a high-quality execution environment, and a vast library. That combination is what makes Java an irresistible proposition to so many programmers.

1.2 The Java “White Paper” Buzzwords

The authors of Java wrote an influential white paper that explains their design goals and accomplishments. They also published a shorter overview that is organized along the following 11 buzzwords:

1. Simple
2. Object-Oriented
3. Distributed
4. Robust
5. Secure
6. Architecture-Neutral

7. Portable
8. Interpreted
9. High-Performance
10. Multithreaded
11. Dynamic

In the following subsections, you will find a summary, with excerpts from the white paper, of what the Java designers say about each buzzword, together with a commentary based on my experiences with the current version of Java.



NOTE: The white paper can be found at www.oracle.com/technetwork/java/langenv-140151.html. You can retrieve the overview with the 11 buzzwords at <http://horstmann.com/corejava/java-an-overview/7Gosling.pdf>.

1.2.1 Simple

We wanted to build a system that could be programmed easily without a lot of esoteric training and which leveraged today's standard practice. So even though we found that C++ was unsuitable, we designed Java as closely to C++ as possible in order to make the system more comprehensible. Java omits many rarely used, poorly understood, confusing features of C++ that, in our experience, bring more grief than benefit.

The syntax for Java is, indeed, a cleaned-up version of C++ syntax. There is no need for header files, pointer arithmetic (or even a pointer syntax), structures, unions, operator overloading, virtual base classes, and so on. (See the C++ notes interspersed throughout the text for more on the differences between Java and C++.) The designers did not, however, attempt to fix all of the clumsy features of C++. For example, the syntax of the `switch` statement is unchanged in Java. If you know C++, you will find the transition to the Java syntax easy.

At the time Java was released, C++ was actually not the most commonly used programming language. Many developers used Visual Basic and its drag-and-drop programming environment. These developers did not find Java simple. It took several years for Java development environments to catch up. Nowadays, Java development environments are far ahead of those for most other programming languages.

Another aspect of being simple is being small. One of the goals of Java is to enable the construction of software that can run stand-alone on small machines. The size of the basic interpreter and class support is about 40K; the basic standard libraries and thread support (essentially a self-contained microkernel) add another 175K.

This was a great achievement at the time. Of course, the library has since grown to huge proportions. There are now separate editions with a smaller library, suitable for embedded devices and smart cards.

1.2.2 Object-Oriented

Simply stated, object-oriented design is a programming technique that focuses on the data—objects—and on the interfaces to those objects. To make an analogy with carpentry, an “object-oriented” carpenter would be mostly concerned with the chair he is building, and secondarily with the tools used to make it; a “non-object-oriented” carpenter would think primarily of his tools. The object-oriented facilities of Java are essentially those of C++.

Object orientation was pretty well established when Java was developed. The object-oriented features of Java are comparable to those of C++. The major difference between Java and C++ lies in multiple inheritance, which Java has replaced with a simpler concept of interfaces. Java has a richer capacity for runtime introspection (discussed in Chapter 5) than C++.

1.2.3 Distributed

Java has an extensive library of routines for coping with TCP/IP protocols like HTTP and FTP. Java applications can open and access objects across the Net via URLs with the same ease as when accessing a local file system.

Nowadays, one takes this for granted—but in 1995, connecting to a web server from a C++ or Visual Basic program was a major undertaking.

1.2.4 Robust

Java is intended for writing programs that must be reliable in a variety of ways. Java puts a lot of emphasis on early checking for possible problems, later dynamic (runtime) checking, and eliminating situations that are error-prone. . . . The single biggest difference between Java and C/C++ is that Java has a pointer model that eliminates the possibility of overwriting memory and corrupting data.

The Java compiler detects many problems that in other languages would show up only at runtime. As for the second point, anyone who has spent hours

chasing memory corruption caused by a pointer bug will be very happy with this aspect of Java.

1.2.5 Secure

Java is intended to be used in networked/distributed environments. Toward that end, a lot of emphasis has been placed on security. Java enables the construction of virus-free, tamper-free systems.

From the beginning, Java was designed to make certain kinds of attacks impossible, among them:

- Overrunning the runtime stack—a common attack of worms and viruses
- Corrupting memory outside its own process space
- Reading or writing files without permission

Originally, the Java attitude towards downloaded code was “Bring it on!” Untrusted code was executed in a sandbox environment where it could not impact the host system. Users were assured that nothing bad could happen because Java code, no matter where it came from, could never escape from the sandbox.

However, the security model of Java is complex. Not long after the first version of the Java Development Kit was shipped, a group of security experts at Princeton University found subtle bugs that allowed untrusted code to attack the host system.

Initially, security bugs were fixed quickly. Unfortunately, over time, hackers got quite good at spotting subtle flaws in the implementation of the security architecture. Sun, and then Oracle, had a tough time keeping up with bug fixes.

After a number of high-profile attacks, browser vendors and Oracle became increasingly cautious. For a time, remote code had to be digitally signed. Nowadays, delivering Java applications via a browser is a distant memory.



NOTE: Even though in hindsight, the Java security model was not as successful as originally envisioned, Java was well ahead of its time. A competing code delivery mechanism from Microsoft, called ActiveX, relied on digital signatures alone for security. Clearly this was not sufficient: As any user of Microsoft’s own products can confirm, programs from well-known vendors do crash and create damage.

1.2.6 Architecture-Neutral

The compiler generates an architecture-neutral object file format. The compiled code is executable on many processors, given the presence of the Java runtime system. The Java compiler does this by generating bytecode instructions which have nothing to do with a particular computer architecture. Rather, they are designed to be both easy to interpret on any machine and easy to translate into native machine code on the fly.

Generating code for a “virtual machine” was not a new idea at the time. Programming languages such as Lisp, Smalltalk, and Pascal had employed this technique for many years.

Of course, interpreting virtual machine instructions is slower than running machine instructions at full speed. However, virtual machines have the option of translating the most frequently executed bytecode sequences into machine code—a process called just-in-time compilation.

Java’s virtual machine has another advantage. It increases security because it can check the behavior of instruction sequences.

1.2.7 Portable

Unlike C and C++, there are no “implementation-dependent” aspects of the specification. The sizes of the primitive data types are specified, as is the behavior of arithmetic on them.

For example, an `int` in Java is always a 32-bit integer. In C/C++, `int` can mean a 16-bit integer, a 32-bit integer, or any other size that the compiler vendor likes. The only restriction is that the `int` type must have at least as many bytes as a `short int` and cannot have more bytes than a `long int`. Having a fixed size for number types eliminates a major porting headache. Binary data is stored and transmitted in a fixed format, eliminating confusion about byte ordering. Strings are saved in a standard Unicode format.

The libraries that are a part of the system define portable interfaces. For example, there is an abstract `Window` class and implementations of it for UNIX, Windows, and the Macintosh.

The example of a `Window` class was perhaps poorly chosen. As anyone who has ever tried knows, it is an effort of heroic proportions to implement a user interface that looks good on Windows, the Macintosh, and ten flavors of UNIX. Java 1.0 made the heroic effort, delivering a simple toolkit that provided common user interface elements on a number of platforms. Unfortunately, the result was a library that, with a lot of work, could give barely acceptable

results on different systems. That initial user interface toolkit has since been replaced, and replaced again, and portability across platforms remains an issue.

However, for everything that isn’t related to user interfaces, the Java libraries do a great job of letting you work in a platform-independent manner. You can work with files, regular expressions, XML, dates and times, databases, network connections, threads, and so on, without worrying about the underlying operating system. Not only are your programs portable, but the Java APIs are often of higher quality than the native ones.

1.2.8 Interpreted

The Java interpreter can execute Java bytecodes directly on any machine to which the interpreter has been ported. Since linking is a more incremental and lightweight process, the development process can be much more rapid and exploratory.

This was a real stretch. Anyone who has used Lisp, Smalltalk, Visual Basic, Python, R, or Scala knows what a “rapid and exploratory” development process is. You try out something, and you instantly see the result. For the first 20 years of Java’s existence, development environments were not focused on that experience. It wasn’t until Java 9 that the `jshell` tool supported rapid and exploratory programming.

1.2.9 High-Performance

While the performance of interpreted bytecodes is usually more than adequate, there are situations where higher performance is required. The bytecodes can be translated on the fly (at runtime) into machine code for the particular CPU the application is running on.

In the early years of Java, many users disagreed with the statement that the performance was “more than adequate.” Today, however, the just-in-time compilers have become so good that they are competitive with traditional compilers and, in some cases, even outperform them because they have more information available. For example, a just-in-time compiler can monitor which code is executed frequently and optimize just that code for speed. A more sophisticated optimization is the elimination (or “inlining”) of function calls. The just-in-time compiler knows which classes have been loaded. It can use inlining when, based upon the currently loaded collection of classes, a particular function is never overridden, and it can undo that optimization later if necessary.

1.2.10 Multithreaded

[The] benefits of multithreading are better interactive responsiveness and real-time behavior.

Nowadays, we care about concurrency because Moore's law has come to an end. Instead of faster processors, we just get more of them, and we have to keep them busy. Yet when you look at most programming languages, they show a shocking disregard for this problem.

Java was well ahead of its time. It was the first mainstream language to support concurrent programming. As you can see from the white paper, its motivation was a little different. At the time, multicore processors were exotic, but web programming had just started, and processors spent a lot of time waiting for a response from the server. Concurrent programming was needed to make sure the user interface didn't freeze.

Concurrent programming is never easy, but Java has done a very good job making it manageable.

1.2.11 Dynamic

In a number of ways, Java is a more dynamic language than C or C++. It was designed to adapt to an evolving environment. Libraries can freely add new methods and instance variables without any effect on their clients. In Java, finding out runtime type information is straightforward.

This is an important feature in situations where code needs to be added to a running program. A prime example is code that is downloaded from the Internet to run in a browser. In C or C++, this is indeed a major challenge, but the Java designers were well aware of dynamic languages that made it easy to evolve a running program. Their achievement was to bring this feature to a mainstream programming language.



NOTE: Shortly after the initial success of Java, Microsoft released a product called J++ with a programming language and virtual machine that were almost identical to Java. This effort failed to gain traction, and Microsoft followed through with another language called C# that also has many similarities to Java but runs on a different virtual machine. This book does not cover J++ or C#.

1.3 Java Applets and the Internet

The idea here is simple: Users will download Java bytecodes from the Internet and run them on their own machines. Java programs that work on web pages are called *applets*. To use an applet, you only need a Java-enabled web browser, which will execute the bytecodes for you. You need not install any software. You get the latest version of the program whenever you visit the web page containing the applet. Most importantly, thanks to the security of the virtual machine, you never need to worry about attacks from hostile code.

Inserting an applet into a web page works much like embedding an image. The applet becomes a part of the page, and the text flows around the space used for the applet. The point is, this image is *alive*. It reacts to user commands, changes its appearance, and exchanges data between the computer presenting the applet and the computer serving it.

Figure 1.1 shows the Jmol applet that displays molecular structures. By using the mouse, you can rotate and zoom each molecule to better understand its structure. At the time that applets were invented, this kind of direct manipulation was not achievable with web pages—there was only rudimentary JavaScript and no HTML canvas.

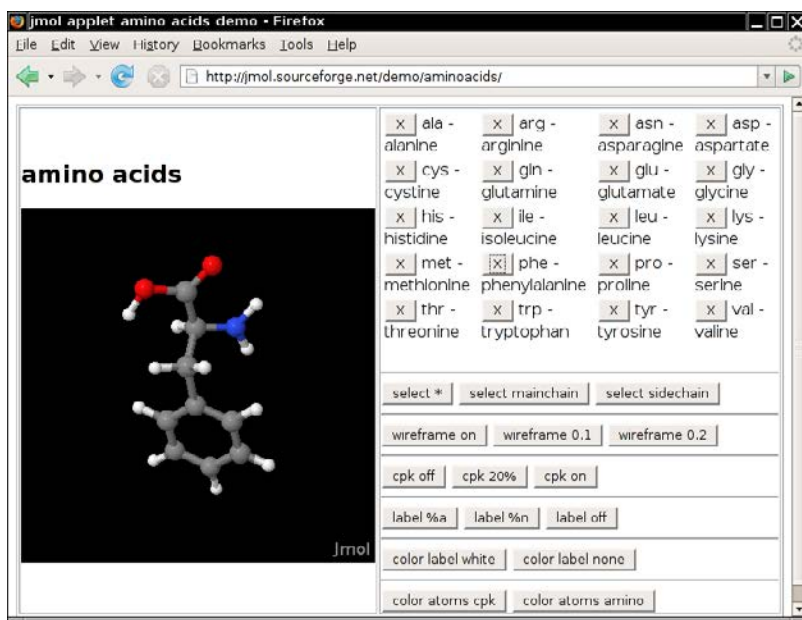


Figure 1.1 The Jmol applet

When applets first appeared, they created a huge amount of excitement. Many people believe that the lure of applets was responsible for the astonishing popularity of Java. However, the initial excitement soon turned into frustration. Various versions of the Netscape and Internet Explorer browsers ran different versions of Java, some of which were seriously outdated. This sorry situation made it increasingly difficult to develop applets that took advantage of the most current Java version. Instead, Adobe's Flash technology became popular for achieving dynamic effects in the browser. Later, when Java was dogged by serious security issues, browsers dropped applet support altogether. Of course, Flash fared no better.

1.4 A Short History of Java

This section gives a short history of Java's evolution. It is based on various published sources (most importantly an interview with Java's creators in the July 1995 issue of *SunWorld's* online magazine).

Java goes back to 1991, when a group of Sun engineers, led by Patrick Naughton and James Gosling (a Sun Fellow and an all-around computer wizard), wanted to design a small computer language that could be used for consumer devices like cable TV switchboxes. Since these devices do not have a lot of power or memory, the language had to be small and generate very tight code. Also, as different manufacturers may choose different central processing units (CPUs), it was important that the language not be tied to any single architecture. The project was code-named "Green."

The requirements for small, tight, and platform-neutral code led the team to design a portable language that generated intermediate code for a virtual machine.

The Sun people came from a UNIX background, so they based their language on C++ rather than Lisp, Smalltalk, or Pascal. But, as Gosling says in the interview, "All along, the language was a tool, not the end." Gosling decided to call his language "Oak" (presumably because he liked the look of an oak tree that was right outside his window at Sun). The people at Sun later realized that Oak was the name of an existing computer language, so they changed the name to Java. This turned out to be an inspired choice.

In 1992, the Green project delivered its first product, called "7." It was an extremely intelligent remote control. Unfortunately, no one was interested in producing this at Sun, and the Green people had to find other ways to market their technology. However, none of the standard consumer electronics companies were interested either. The group then bid on a project to design a cable TV box that could deal with emerging cable services such as video-on-

demand. They did not get the contract. (Amusingly, the company that did was led by the same Jim Clark who started Netscape—a company that did much to make Java successful.)

The Green project (with a new name of “First Person, Inc.”) spent all of 1993 and half of 1994 looking for people to buy its technology. No one was found. (Patrick Naughton, one of the founders of the group and the person who ended up doing most of the marketing, claims to have accumulated 300,000 air miles in trying to sell the technology.) First Person was dissolved in 1994.

While all of this was going on at Sun, the World Wide Web part of the Internet was growing bigger and bigger. The key to the World Wide Web was the browser translating hypertext pages to the screen. In 1994, most people were using Mosaic, a noncommercial web browser that came out of the supercomputing center at the University of Illinois in 1993. (Mosaic was partially written by Marc Andreessen as an undergraduate student on a work-study project, for \$6.85 an hour. He moved on to fame and fortune as one of the cofounders and the chief of technology at Netscape.)

In the *SunWorld* interview, Gosling says that in mid-1994, the language developers realized that “We could build a real cool browser. It was one of the few things in the client/server mainstream that needed some of the weird things we’d done: architecture-neutral, real-time, reliable, secure—issues that weren’t terribly important in the workstation world. So we built a browser.”

The actual browser was built by Patrick Naughton and Jonathan Payne and evolved into the HotJava browser, which was designed to show off the power of Java. The browser was capable of executing Java code inside web pages. This “proof of technology” was shown at SunWorld ’95 on May 23, 1995, and inspired the Java craze that continues today.

Sun released the first version of Java in early 1996. People quickly realized that Java 1.0 was not going to cut it for serious application development. Sure, you could use Java 1.0 to make a nervous text applet that moved text randomly around in a canvas. But you couldn’t even *print* in Java 1.0. To be blunt, Java 1.0 was not ready for prime time. Its successor, version 1.1, filled in the most obvious gaps, greatly improved the reflection capability, and added a new event model for GUI programming. It was still rather limited, though.

The big news of the 1998 JavaOne conference was the upcoming release of Java 1.2, which replaced the early toylike GUI and graphics toolkits with sophisticated scalable versions. Three days (!) after its release in December 1998, Sun’s marketing department changed the name to the catchy *Java 2 Standard Edition Software Development Kit Version 1.2*.

Besides the Standard Edition, two other editions were introduced: the Micro Edition for embedded devices such as cell phones, and the Enterprise Edition for server-side processing. This book focuses on the Standard Edition.

Versions 1.3 and 1.4 of the Standard Edition were incremental improvements over the initial Java 2 release, with an ever-growing standard library, increased performance, and, of course, quite a few bug fixes. During this time, much of the initial hype about Java applets and client-side applications abated, but Java became the platform of choice for server-side applications.

Version 5.0 was the first release since version 1.1 that updated the Java *language* in significant ways. (This version was originally numbered 1.5, but the version number jumped to 5.0 at the 2004 JavaOne conference.) After many years of research, generic types (roughly comparable to C++ templates) have been added—the challenge was to add this feature without requiring changes in the virtual machine. Several other useful language features were inspired by C#: a “for each” loop, autoboxing, and annotations.

Version 6 (without the .0 suffix) was released at the end of 2006. Again, there were no language changes but additional performance improvements and library enhancements.

As datacenters increasingly relied on commodity hardware instead of specialized servers, Sun Microsystems fell on hard times and was purchased by Oracle in 2009. Development of Java stalled for a long time. In 2011, Oracle released a new version, with simple enhancements, as Java 7.

In 2014, the release of Java 8 followed, with the most significant changes to the Java language in almost two decades. Java 8 embraces a “functional” style of programming that makes it easy to express computations that can be executed concurrently. All programming languages must evolve to stay relevant, and Java has shown a remarkable capacity to do so.

The main feature of Java 9 goes all the way back to 2008. At that time, Mark Reinhold, the chief engineer of the Java platform, started an effort to break up the huge, monolithic Java platform. This was to be achieved by introducing *modules*, self-contained units of code that provide a specific functionality. It took eleven years to design and implement a module system that is a good fit for the Java platform, and it remains to be seen whether it is also a good fit for Java applications and libraries. Java 9, released in 2017, has other appealing features that are covered in this book.

Starting in 2018, Java versions are released every six months, to enable faster introduction of features. Ever so often, a version—so far Java 11 and Java 17—is designated as a long-term support version. Intermediate versions provide a mechanism for trying out experimental features.

Table 1.1 shows the evolution of the Java language and library. As you can see, the size of the application programming interface (API) has grown tremendously.

Table 1.1 Evolution of the Java Language

Version	Year	New Language Features	Number of Classes and Interfaces
1.0	1996	The language itself	211
1.1	1997	Inner classes	477
1.2	1998	The <code>strictfp</code> modifier	1,524
1.3	2000	None	1,840
1.4	2002	Assertions	2,723
5.0	2004	Generic classes, “for each” loop, varargs, autoboxing, metadata, enumerations, static import	3,279
6	2006	None	3,793
7	2011	Switch with strings, diamond operator, binary literals, exception handling enhancements	4,024
8	2014	Lambda expressions, interfaces with default methods, stream and date/time libraries	4,240
9	2017	Modules, miscellaneous language and library enhancements	6,005
11	2018	Local variable type inference (<code>var</code>), HTTP client, removal of Java FX, JNLP, Java EE overlap, and CORBA	4,410
17	2021	Switch expressions, text blocks, instanceof pattern matching, records, sealed classes	4,859

1.5 Common Misconceptions about Java

This chapter closes with a commented list of some common misconceptions about Java.

Java is an extension of HTML.

Java is a programming language; HTML is a way to describe the structure of a web page. They have nothing in common except that there once were HTML extensions for placing Java applets on a web page.

I use XML, so I don't need Java.

Java is a programming language; XML is a way to describe data. You can process XML data with any programming language, but the Java API contains excellent support for XML processing. In addition, many important XML tools are implemented in Java. See Volume II for more information.

Java is an easy programming language to learn.

No programming language as powerful as Java is easy. You always have to distinguish between how easy it is to write toy programs and how hard it is to do serious work. Also, consider that only seven chapters in this book discuss the Java language. The remaining chapters of both volumes show how to put the language to work, using the Java *libraries*. The Java libraries contain thousands of classes and interfaces and tens of thousands of functions. Luckily, you do not need to know every one of them, but you do need to know surprisingly many to use Java for anything realistic.

Java will become a universal programming language for all platforms.

This is possible in theory. But in practice, there are domains where other languages are entrenched. Objective C and its successor, Swift, are not going to be replaced on iOS devices. Anything that happens in a browser is controlled by JavaScript. Windows programs are written in C++ or C#. Java has the edge in server-side programming and in cross-platform client applications.

Java is just another programming language.

Java is a nice programming language; most programmers prefer it to C, C++, or C#. But there have been hundreds of nice programming languages that never gained widespread popularity, whereas languages with obvious flaws, such as C++ and Visual Basic, have been wildly successful.

Why? The success of a programming language is determined far more by the utility of the *support system* surrounding it than by the elegance of its syntax. Are there useful, convenient, and standard libraries for the features that you need to implement? Are there tool vendors that build great programming and debugging environments? Do the language and the toolset integrate with the rest of the computing infrastructure? Java is successful because its libraries let you easily do things such as networking, web applications, and concurrency.

The fact that Java reduces pointer errors is a bonus, so programmers seem to be more productive with Java—but these factors are not the source of its success.

Java is proprietary, and should therefore be avoided.

When Java was first created, Sun Microsystems gave free licenses to distributors and end users. Although Sun had ultimate control over Java, they involved many other companies in the development of language revisions and the design of new libraries. Source code for the virtual machine and the libraries has always been freely available, but only for inspection, not for modification and redistribution. Java was “closed source, but playing nice.”

This situation changed dramatically in 2007, when Sun announced that future versions of Java would be available under the General Public License (GPL), the same open source license that is used by Linux. Oracle has committed to keeping Java open source. There are now multiple providers of open Java implementations, with various levels of commitment and support.

Java is interpreted, so it is too slow for serious applications.

In the early days of Java, the language was interpreted. Nowadays, the Java virtual machine uses a just-in-time compiler. The “hot spots” of your code will run just as fast in Java as they would in C++, and in some cases even faster.

All Java programs run inside a web page.

There was a time when Java *applets* ran inside a web browser. Nowadays, Java programs are stand-alone applications that run outside of a web browser. In fact, most Java programs run on servers, producing code for web pages or computing business logic.

Java programs are a major security risk.

In the early days of Java, there were some well-publicized reports of failures in the Java security system. Researchers viewed it as a challenge to find chinks in the Java armor and to defy the strength and sophistication of the applet security model. The technical failures that they found had been quickly corrected. Later, there were more serious exploits, to which Sun, and later Oracle, responded slowly. Browser manufacturers discontinued support for Java applets. The security manager architecture that made applets possible is now deprecated. These days, Java applications are no less secure than other applications. Due to the protections of the virtual machine, they are far more secure than applications written in C or C++.

JavaScript is a simpler version of Java.

JavaScript, a scripting language that can be used inside web pages, was invented by Netscape and originally called LiveScript. JavaScript has a syntax that is reminiscent of Java, and the languages' names sound similar, but otherwise they are unrelated. In particular, Java is *strongly typed*—the compiler catches many errors that arise from type misuse. In JavaScript, such errors are only found when the program runs, which makes their elimination far more laborious.

With Java, I can replace my desktop computer with a cheap “Internet appliance.”

When Java was first released, some people bet big that this was going to happen. Companies produced prototypes of Java-powered network computers, but users were not ready to give up a powerful and convenient desktop for a limited machine with no local storage. Nowadays, of course, the world has changed, and for a large majority of end users, the platform that matters is a mobile phone or tablet. The majority of these devices are controlled by the Android platform. Learning Java programming will help you with Android programming as well.

The Java Programming Environment

In this chapter

- 2.1 Installing the Java Development Kit, page 18
- 2.2 Using the Command-Line Tools, page 22
- 2.3 Using an Integrated Development Environment, page 27
- 2.4 JShell, page 30

In this chapter, you will learn how to install the Java Development Kit (JDK) and how to compile and run Java programs. You can run the JDK tools by typing commands in a terminal window. However, many programmers prefer the comfort of an integrated development environment. You will learn how to use a freely available development environment to compile and run Java programs. Once you have mastered the techniques in this chapter and picked your development tools, you are ready to move on to Chapter 3, where you will begin exploring the Java programming language.

2.1 Installing the Java Development Kit

There are many organizations that make the Java Development Kit (JDK) available for free in addition to Oracle. This includes Amazon, Azul, Eclipse, Microsoft, Red Hat and others.

2.1.1 Downloading the JDK

You can download the Oracle JDK from oracle.com/javadownload, OpenJDK from jdk.java.net, or the JDK from other parties via their respective download sites. You should use the Java SE 17 (LTS) JDK. See Table 2.1 for a summary of the acronyms and jargon that you may encounter on download sites.

Table 2.1 Java Jargon

Name	Acronym	Explanation
Java Development Kit	JDK	The software for programmers who want to write Java programs.
Java Runtime Environment	JRE	The software for running Java programs, without development tools. You do not want that.
Standard Edition	SE	The Java platform for use on desktops and simple server applications. You want that.
Micro Edition	ME	The Java platform for use on small devices.
OpenJDK	—	A free and open source implementation of Java SE.
Hotspot	—	The “just in time” compiler developed by Oracle. If asked, choose this one.
OpenJ9	—	Another “just in time” compiler developed by IBM.
Long Term Support	LTS	A release that is supported for multiple years, unlike the six-month releases that showcase new features. Choose the latest LTS release.

2.1.2 Setting Up the JDK

After downloading the JDK, you need to install it and figure out where it was installed—you’ll need that information later.

- Under Windows, launch the setup program. You will be asked where to install the JDK. It is best not to accept a default location with spaces in

the path name, such as `c:\Program Files\Java\jdk-17.0.x`. Just take out the Program Files part of the path name.

- On the Mac, run the installer. It installs the software into `/Library/Java/JavaVirtualMachines/jdk-17.0.x.jdk/Contents/Home`. Locate it with the Finder.
- On Linux, simply uncompress the `.tar.gz` file to a location of your choice, such as your home directory or `/opt`. Or, if you installed from the RPM file, double-check that it is installed in `/usr/java/jdk-17.0.x`.

In this book, the installation directory is denoted as *jdk*. For example, when referring to the *jdk/bin* directory, I mean the directory with a name such as `/opt/jdk-17.0.4/bin` or `c:\Java\jdk-17.0.4\bin`.

Here is how you test whether you did it right. Start a terminal window. Type the line

```
javac --version
```

and press the Enter key. You should get a display such as this one:

```
javac 17.0.4
```

If instead you get a message such as “javac: command not found” or “The name specified is not recognized as an internal or external command, operable program or batch file,” then you need to double-check your installation.

When you install the JDK on Windows or Linux, you may need to carry out one additional step: Add the *jdk/bin* directory to the executable path—the list of directories that the operating system traverses to locate executable files.

- On Linux, add a line such as the following to the end of your `~/.bashrc` or `~/.bash_profile` file:

```
export PATH=jdk/bin:$PATH
```

Be sure to use the correct path to the JDK, such as `/opt/jdk-17.0.4`.

- Under Windows 10, type “environment” into the search bar of the Windows Settings, and select “Edit environment variables for your account” (see Figure 2.1). An Environment Variables dialog should appear. (It may hide behind the Windows Settings dialog. If you can’t find it anywhere, try running `sysdm.cpl` from the Run dialog that you get by holding down the Windows and R key at the same time, and then select the Advanced tab and click the Environment Variables button.) Locate and select a variable named Path in the User Variables list. Click the Edit button, then the New button, and add an entry with the *jdk\bin* directory (see Figure 2.2).

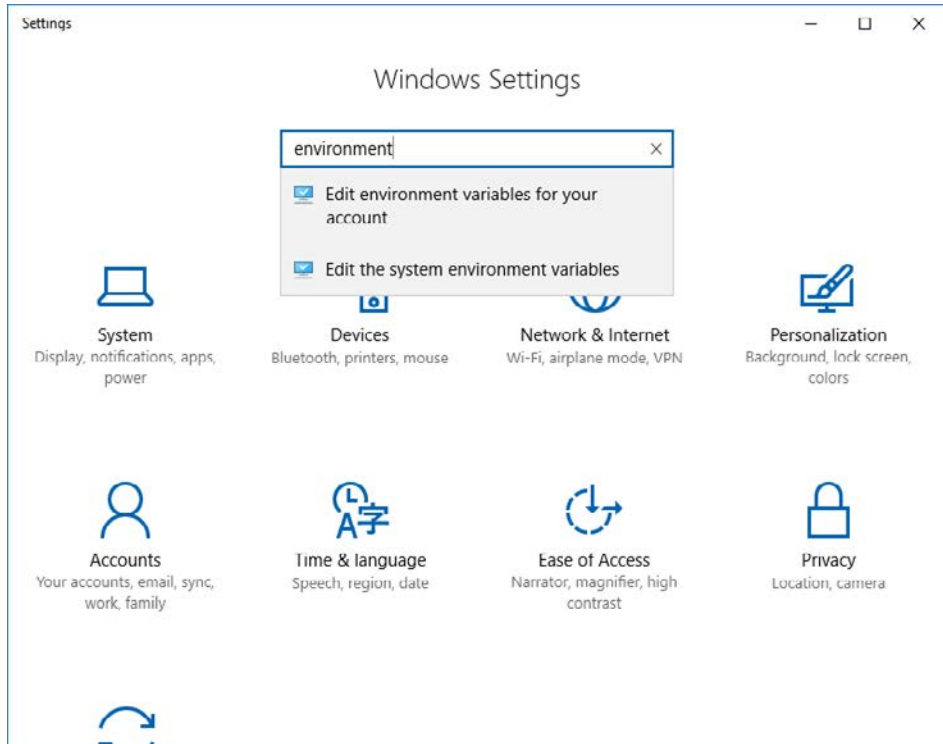


Figure 2.1 Setting system properties in Windows

Save your settings. Any new “Command Prompt” windows that you start will have the correct path.

2.1.3 Installing Source Files and Documentation

The library source files are delivered in the JDK as a compressed file `lib/src.zip`. Unpack that file to get access to the source code. Simply do the following:

1. Make sure the JDK is installed and the `jdk/bin` directory is on the executable path.
2. Make a directory `javasrc` in your home directory. If you like, you can do this from a terminal window.

```
mkdir javasrc
```
3. Inside the `jdk/lib` directory, locate the file `src.zip`.
4. Unzip the `src.zip` file into the `javasrc` directory. In a terminal window, you can execute the commands

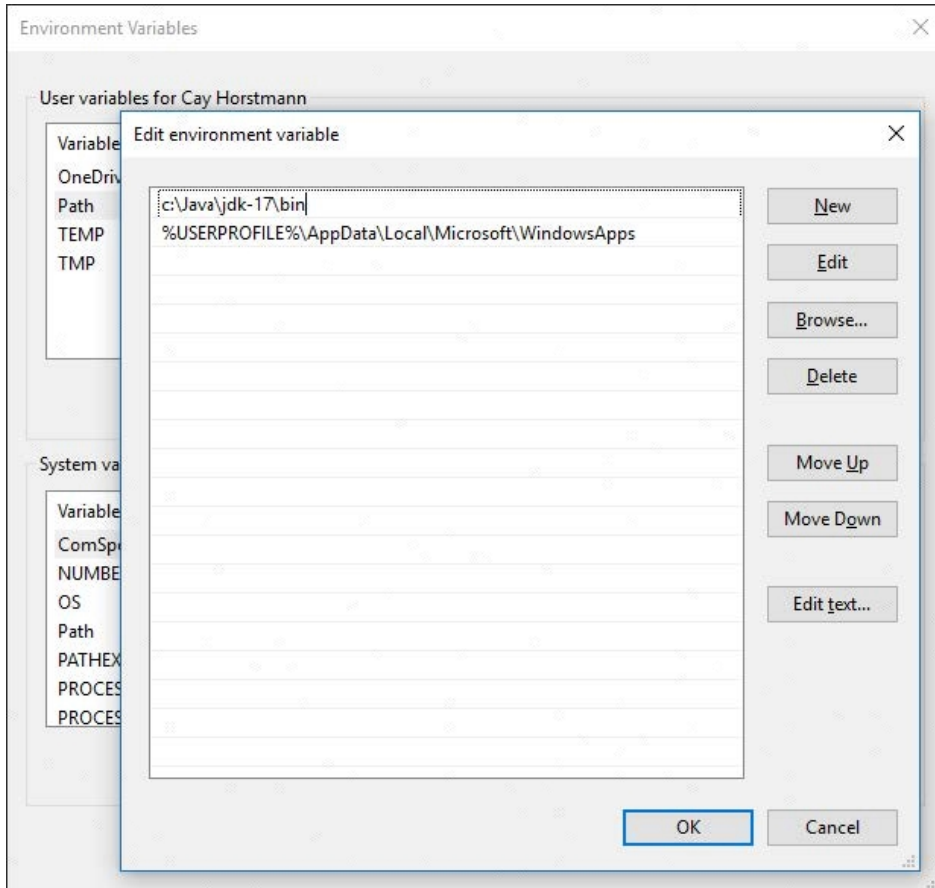


Figure 2.2 Setting the Path environment variable in Windows

```
cd javasrc
jar xvf jdk/lib/src.zip
cd ..
```



TIP: The `src.zip` file contains the source code for all public libraries. To obtain even more source (for the compiler, the virtual machine, the native methods, and the private helper classes), go to <http://openjdk.java.net>.

The documentation is contained in a compressed file that is separate from the JDK. You can download the documentation from www.oracle.com/technetwork/java/javase/downloads. Follow these steps:

1. Download the documentation zip file. It is called `jdk-17.0.x_doc-all.zip`.
2. Unzip the file and rename the `doc` directory into something more descriptive, like `javadoc`. If you like, you can do this from the command line:

```
jar xvf Downloads/jdk-17.0.x_doc-all.zip
mv docs jdk-17-docs
```

3. In your browser, navigate to `jdk-17-docs/index.html` and add this page to your bookmarks.

You should also install the *Core Java* program examples. You can download them from <http://horstmann.com/corejava>. The programs are packaged into a zip file `corejava.zip`. Just unzip them into your home directory. They will be located in a directory `corejava`. If you like, you can do this from the command line:

```
jar xvf Downloads/corejava.zip
```

2.2 Using the Command-Line Tools

If your programming experience comes from a development environment such as Microsoft Visual Studio, you are accustomed to a system with a built-in text editor, menus to compile and launch a program, and a debugger. The JDK contains nothing even remotely similar. You do *everything* by typing in commands in a terminal window. This may sound cumbersome, but it is nevertheless an essential skill. When you first install Java, you will want to troubleshoot your installation before you install a development environment. Moreover, by executing the basic steps yourself, you gain a better understanding of what a development environment does behind your back.

However, after you have mastered the basic steps of compiling and running Java programs, you will want to use a professional development environment. You will see how to do that in the following section.

Let's get started the hard way: compiling and launching a Java program from the command line.

1. Open a terminal window.
2. Go to the `corejava/v1ch02/Welcome` directory. (The `corejava` directory is where you installed the source code for the book examples, as explained in Section 2.1.3, "Installing Source Files and Documentation," on p. 20.)
3. Enter the following commands:

```
javac Welcome.java
java Welcome
```

You should see the output shown in Figure 2.3 in the terminal window.

A screenshot of a terminal window titled "Terminal". The window shows the following commands and output:

```
~$ cd corejava/vlch02/Welcome
~/corejava/vlch02/Welcome$ javac Welcome.java
~/corejava/vlch02/Welcome$ java Welcome
Welcome to Core Java!
=====
~/corejava/vlch02/Welcome$
```

Figure 2.3 Compiling and running `Welcome.java`

Congratulations! You have just compiled and run your first Java program.

What happened? The `javac` program is the Java compiler. It compiles the file `Welcome.java` into the file `Welcome.class`. The `java` program launches the Java virtual machine. It executes the bytecodes that the compiler placed in the class file.

The `Welcome` program is extremely simple. It merely prints a message to the terminal. You may enjoy looking inside the program, shown in Listing 2.1. You will see how it works in the next chapter.

Listing 2.1 `Welcome/Welcome.java`

```
1 /**
2  * This program displays a greeting for the reader.
3  * @version 1.30 2014-02-27
4  * @author Cay Horstmann
5  */
6 public class Welcome
7 {
8     public static void main(String[] args)
9     {
10         String greeting = "Welcome to Core Java!";
11         System.out.println(greeting);
12         for (int i = 0; i < greeting.length(); i++)
13             System.out.print("=");
14         System.out.println();
15     }
16 }
```

In the age of integrated development environments, many programmers are unfamiliar with running programs in a terminal window. Any number of things can go wrong, leading to frustrating results.

Pay attention to the following points:

- If you type in the program by hand, make sure you correctly enter the uppercase and lowercase letters. In particular, the class name is `Welcome` and not `welcome` or `WELCOME`.
- The compiler requires a *file name* (`Welcome.java`). When you run the program, you specify a *class name* (`Welcome`) without a `.java` or `.class` extension.
- If you get a message such as “Bad command or file name” or “javac: command not found,” go back and double-check your installation, in particular the executable path setting.
- If `javac` reports that it cannot find the file `Welcome.java`, you should check whether that file is present in the directory.

Under Linux, check that you used the correct capitalization for `Welcome.java`.

Under Windows, use the `dir` command, *not* the graphical Explorer tool. Some text editors (in particular Notepad) insist on adding an extension `.txt` to every file’s name. If you use Notepad to edit `Welcome.java`, it will actually save it as `Welcome.java.txt`. Under the default Windows settings, Explorer conspires with Notepad and hides the `.txt` extension because it belongs to a “known file type.” In that case, you need to rename the file, using the `ren` command, or save it again, placing quotes around the file name: “`Welcome.java`”.

- If you launch your program and get an error message complaining about a `java.lang.NoClassDefFoundError`, then carefully check the name of the offending class.

If you get a complaint about `welcome` (with a lowercase `w`), then you should reissue the `java Welcome` command with an uppercase `W`. As always, case matters in Java.

If you get a complaint about `Welcome/java`, it means you accidentally typed `java Welcome.java`. Reissue the command as `java Welcome`.

- If you typed `java Welcome` and the virtual machine can’t find the `Welcome` class, check if someone has set the `CLASSPATH` environment variable on your system. It is not a good idea to set this variable globally, but some poorly written software installers in Windows do just that. Follow the same procedure as for setting the `Path` environment variable, but this time, remove the setting.



TIP: The excellent tutorial at <http://docs.oracle.com/javase/tutorial/getStarted/cupojava> goes into much greater detail about the “gotchas” that beginners can run into.



NOTE: You can skip the `javac` command if you have a single source file. This feature is intended for shell scripts (starting with a “shebang” line `#!/path/to/java`) and perhaps for simple student programs. Once your programs get more complex, you need to use `javac`.

The `Welcome` program was not terribly exciting. Next, try out a graphical application. This program is a simple image file viewer that loads and displays an image. As before, compile and run the program from the command line.

1. Open a terminal window.
2. Change to the directory `corejava/v1ch02/ImageViewer`.
3. Enter the following:

```
javac ImageViewer.java
java ImageViewer
```

A new program window pops up with the `ImageViewer` application. Now, select `File` → `Open` and look for an image file to open. (There are a couple of sample files in the same directory.) The image is displayed (see Figure 2.4). To close the program, click on the Close box in the title bar or select `File` → `Exit` from the menu.



Figure 2.4 Running the `ImageViewer` application

Have a quick look at the source code (Listing 2.2). The program is substantially longer than the first program, but it is not too complex if you consider how much code it would take in C or C++ to write a similar application. Of course, nowadays it is not common to write desktop applications with graphical user interfaces, but if you are interested, you can find more details in Chapter 10.

Listing 2.2 ImageViewer/ImageViewer.java

```
1 import java.awt.*;
2 import java.io.*;
3 import javax.swing.*;
4
5 /**
6  * A program for viewing images.
7  * @version 1.31 2018-04-10
8  * @author Cay Horstmann
9  */
10 public class ImageViewer
11 {
12     public static void main(String[] args)
13     {
14         EventQueue.invokeLater(() ->
15             {
16                 var frame = new ImageViewerFrame();
17                 frame.setTitle("ImageViewer");
18                 frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
19                 frame.setVisible(true);
20             });
21     }
22 }
23
24 /**
25  * A frame with a label to show an image.
26  */
27 class ImageViewerFrame extends JFrame
28 {
29     private static final int DEFAULT_WIDTH = 300;
30     private static final int DEFAULT_HEIGHT = 400;
31
32     public ImageViewerFrame()
33     {
34         setSize(DEFAULT_WIDTH, DEFAULT_HEIGHT);
35
36         // use a label to display the images
37         var label = new JLabel();
38         add(label);
39
40         // set up the file chooser
```

```
41     var chooser = new JFileChooser();
42     chooser.setCurrentDirectory(new File("."));
43
44     // set up the menu bar
45     var menuBar = new JMenuBar();
46     setJMenuBar(menuBar);
47
48     var menu = new JMenu("File");
49     menuBar.add(menu);
50
51     var openItem = new JMenuItem("Open");
52     menu.add(openItem);
53     openItem.addActionListener(event ->
54     {
55         // show file chooser dialog
56         int result = chooser.showOpenDialog(null);
57
58         // if file selected, set it as icon of the label
59         if (result == JFileChooser.APPROVE_OPTION)
60         {
61             String name = chooser.getSelectedFile().getPath();
62             label.setIcon(new ImageIcon(name));
63         }
64     });
65
66     var exitItem = new JMenuItem("Exit");
67     menu.add(exitItem);
68     exitItem.addActionListener(event -> System.exit(0));
69 }
70 }
```

2.3 Using an Integrated Development Environment

In the preceding section, you saw how to compile and run a Java program from the command line. That is a useful skill for troubleshooting, but for most day-to-day work, you should use an integrated development environment. These environments are so powerful and convenient that it simply doesn't make much sense to labor on without them. Excellent choices are the freely available Eclipse, IntelliJ IDEA, and NetBeans. In this chapter, you will learn how to get started with Eclipse. Of course, if you prefer a different development environment, you can certainly use it with this book.

Get started by downloading Eclipse from <http://eclipse.org/downloads>. Versions exist for Linux, Mac OS X, and Windows. Run the installation program and pick the installation set called "Eclipse IDE for Java Developers."

Here are the steps to write a program with Eclipse:

1. After starting Eclipse, select File → New → Project from the menu.
2. Select “Java Project” from the wizard dialog (see Figure 2.5).

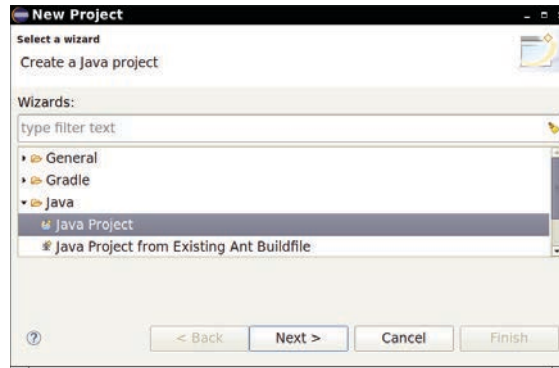


Figure 2.5 The New Project dialog in Eclipse

3. Click the Next button. *Uncheck* the “Use default location” checkbox. Click on Browse and navigate to the corejava/v1ch02/Welcome directory (Figure 2.6).

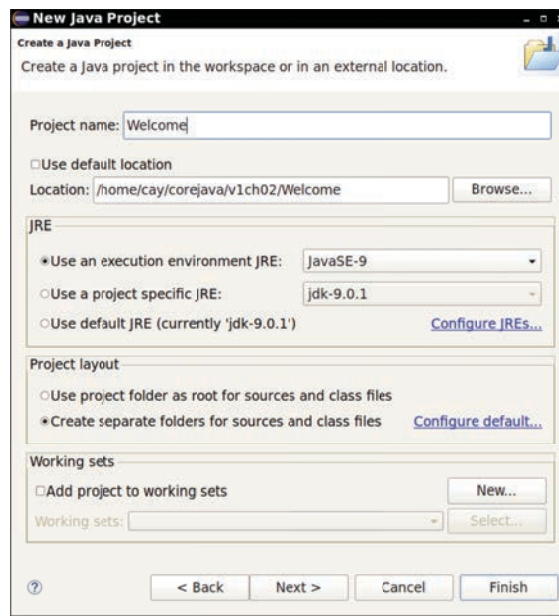


Figure 2.6 Configuring a project in Eclipse

4. Click the Finish button. The project is now created.
5. Click on the triangles in the left pane next to the project until you locate the file `Welcome.java`, and double-click on it. You should now see a pane with the program code (see Figure 2.7).

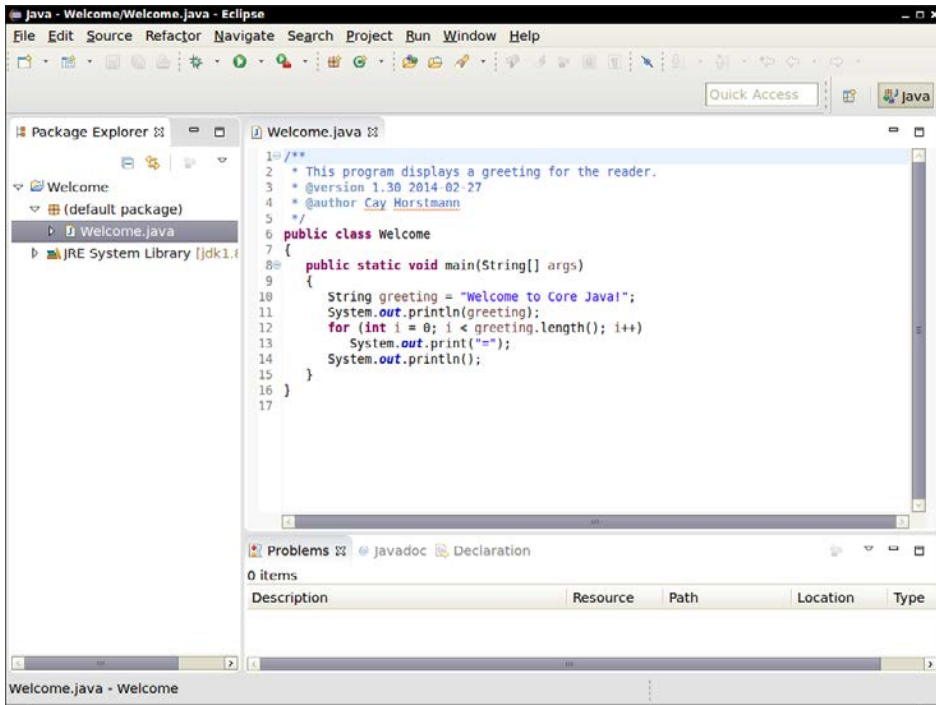


Figure 2.7 Editing a source file with Eclipse

6. With the right mouse button, click on the project name (Welcome) in the left pane. Select Run → Run As → Java Application. The program output is displayed in the console pane.

Presumably, this program does not have typos or bugs. (It was only a few lines of code, after all.) Let us suppose, for the sake of argument, that your code occasionally contains a typo (perhaps even a syntax error). Try it out—ruin your file, for example, by changing the capitalization of `String` as follows:

```
string greeting = "Welcome to Core Java!";
```

Note the wiggly line under `string`. In the tabs below the source code, click on Problems and expand the triangles until you see an error message that complains about an unknown string type (see Figure 2.8). Click on the error message. The cursor moves to the matching line in the edit pane, where you can correct your error. This allows you to fix your errors quickly.

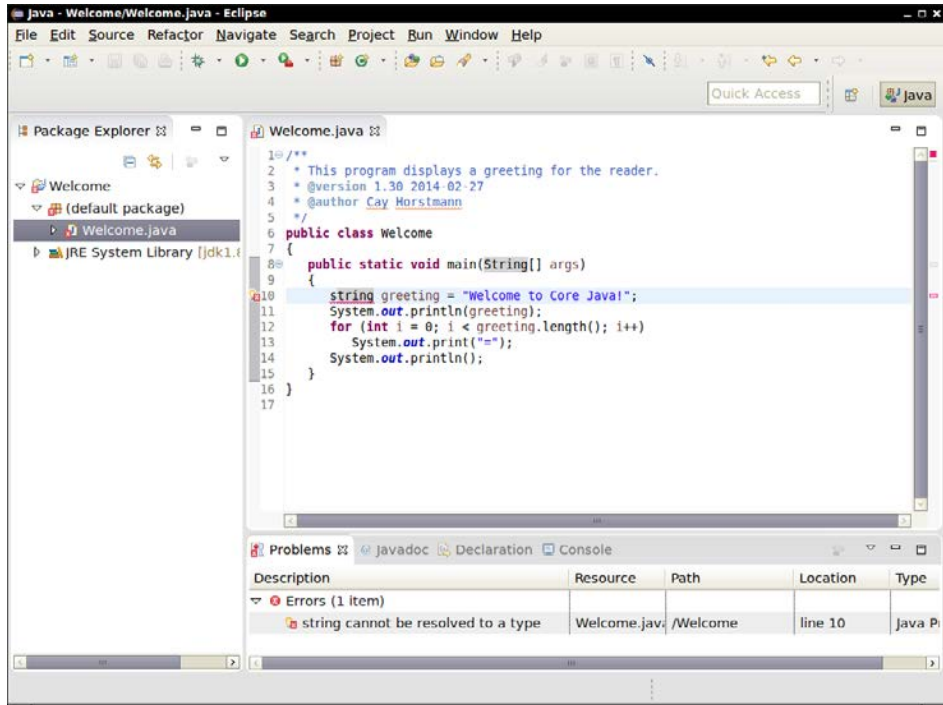


Figure 2.8 Error messages in Eclipse



TIP: Often, an Eclipse error report is accompanied by a lightbulb icon. Click on the lightbulb to get a list of suggested fixes.

2.4 JShell

In the preceding section, you saw how to compile and run a Java program. Java 9 introduced another way of working with Java. The JShell program provides a “read-evaluate-print loop,” or REPL. You type a Java expression; JShell evaluates your input, prints the result, and waits for your next input. To start JShell, simply type `jshell` in a terminal window (see Figure 2.9).

```

Terminal ~$
~$ jshell
| Welcome to JShell -- Version 17
| For an introduction type: /help intro

jshell> "Core Java".length()
$1 ==> 9

jshell> 5 * $1 - 3
$2 ==> 42

jshell> int answer = 6 * 7
answer ==> 42

jshell> Math.
E
absExact(      IEEEremainder(  PI
acos(          addExact(      abs(
atan(          atan2(         cbrt(
class          copySign(     cos(
decrementExact( exp(         expm1(
floorDiv(      floorMod(     fma(
hypot(         incrementExact( log(
log1p(         max(          min(
multiplyFull(  multiplyHigh(  negateExact(
nextDown(      nextUp(        pow(
rint(          round(       scalb(
sin(           sinh(        sqrt(
tan(           tanh(        toDegrees(
toRadians(     ulp(
jshell> Math.

```

Figure 2.9 Running JShell

JShell starts with a greeting, followed by a prompt:

```

| Welcome to JShell -- Version 17.0.4
| For an introduction type: /help intro

```

```
jshell>
```

Now type an expression, such as

```
"Core Java".length()
```

JShell responds with the result—in this case, the number of characters in the string "Core Java".

```
$1 ==> 9
```

Note that you do *not* type `System.out.println`. JShell automatically prints the value of every expression that you enter.

The `$1` in the output indicates that the result is available in further calculations. For example, if you type

```
5 * $1 - 3
```

the response is

```
$2 ==> 42
```

If you need a variable many times, you can give it a more memorable name. You have to follow the Java syntax (explained in Chapter 3) and specify the type, followed by the name. For example,

```
jshell> int answer = 6 * 7
answer ==> 42
```

Another useful feature is tab completion. Type

```
Math.
```

followed by the Tab key. You get a list of all methods that you can invoke with the Math class:

```
jshell> Math.
E               IEEEremainder(  PI               abs(
absExact(       acos(           addExact(         asin(
atan(           atan2(          cbrt(           ceil(
class           copySign(       cos(           cosh(
decrementExact( exp(           expm1(         floor(
floorDiv(       floorMod(       fma(           getExponent(
hypot(          incrementExact( log(           log10(
log1p(          max(           min(           multiplyExact(
multiplyFull(   multiplyHigh(  negateExact(   nextAfter(
nextDown(       nextUp(         pow(           random()
rint(           round(          scalb(         signum(
sin(            sinh(           sqrt(          subtractExact(
tan(            tanh(          toDegrees(     toIntExact(
toRadians(      ulp(
```

Now type `l` and hit the Tab key again. The method name is completed to `log`, and you get a shorter list:

```
jshell> Math.log
log(      log10(  log1p(
```

Now you can fill in the rest by hand:

```
jshell> Math.log10(0.001)
$3 ==> -3.0
```

To repeat a command, hit the `↑` key until you see the line that you want to reissue or edit. You can move the cursor in the line with the `←` and `→` keys, and add or delete characters. Hit Enter when you are done. For example, hit and replace `0.001` with `1000`, then hit Enter:

```
jshell> Math.log10(1000)
$4 ==> 3.0
```

JShell makes it easy and fun to learn about the Java language and library without having to launch a heavy-duty development environment and without fussing with `public static void main`.

In this chapter, you learned about the mechanics of compiling and running Java programs. You are now ready to move on to Chapter 3 where you will start learning the Java language.

This page intentionally left blank

Fundamental Programming Structures in Java

In this chapter

- 3.1 A Simple Java Program, page 36
- 3.2 Comments, page 39
- 3.3 Data Types, page 40
- 3.4 Variables and Constants, page 46
- 3.5 Operators, page 51
- 3.6 Strings, page 61
- 3.7 Input and Output, page 76
- 3.8 Control Flow, page 85
- 3.9 Big Numbers, page 106
- 3.10 Arrays, page 109

At this point, you should have successfully installed the JDK and executed the sample programs from Chapter 2. It's time to start programming. This

chapter shows you how the basic programming concepts such as data types, branches, and loops are implemented in Java.

3.1 A Simple Java Program

Let's look more closely at one of the simplest Java programs you can have—one that merely prints a message to console:

```
public class FirstSample
{
    public static void main(String[] args)
    {
        System.out.println("We will not use 'Hello, World!'");
    }
}
```

It is worth spending all the time you need to become comfortable with the framework of this sample; the pieces will recur in all applications. First and foremost, *Java is case sensitive*. If you made any mistakes in capitalization (such as typing `Main` instead of `main`), the program will not run.

Now let's look at this source code line by line. The keyword `public` is called an *access modifier*; these modifiers control the level of access other parts of a program have to this code. For more about access modifiers, see Chapter 5. The keyword `class` reminds you that everything in a Java program lives inside a class. You will learn a lot more about classes in the next chapter. For now, think of a class as a container for the program logic that defines the behavior of an application. As mentioned in Chapter 1, classes are the building blocks with which all Java applications are built. *Everything* in a Java program must be inside a class.

Following the keyword `class` is the name of the class. The rules for class names in Java are quite generous. Names must begin with a letter, and after that, they can have any combination of letters and digits. The length is essentially unlimited. You cannot use a Java reserved word (such as `public` or `class`) for a class name. (See the appendix for a list of reserved words.)

The standard naming convention (used in the name `FirstSample`) is that class names are nouns that start with an uppercase letter. If a name consists of multiple words, use an initial uppercase letter in each of the words. This use of uppercase letters in the middle of a name is sometimes called “camel case” or, self-referentially, “CamelCase.”

You need to make the file name for the source code the same as the name of the public class, with the extension `.java` appended. Thus, you must store

this code in a file called `FirstSample.java`. (Again, case is important—don't use `firstsample.java`.)

If you have named the file correctly and not made any typos in the source code, then when you compile this source code, you end up with a file containing the bytecodes for this class. The Java compiler automatically names the bytecode file `FirstSample.class` and stores it in the same directory as the source file. Finally, launch the program by issuing the following command:

```
java FirstSample
```

(Remember to leave off the `.class` extension.) When the program executes, it simply displays the string `We will not use 'Hello, World!'` on the console.

When you use

```
java ClassName
```

to run a compiled program, the Java virtual machine always starts execution with the code in the `main` method in the class you indicate. (The term “method” is Java-speak for a function.) Thus, you *must* have a `main` method in the source of your class for your code to execute. You can, of course, add your own methods to a class and call them from the `main` method. (You will see in the next chapter how to write your own methods.)



NOTE: According to the Java Language Specification, the `main` method must be declared `public`. (The Java Language Specification is the official document that describes the Java language. You can view or download it from <http://docs.oracle.com/javase/specs>.)

However, several versions of the Java launcher were willing to execute Java programs even when the `main` method was not `public`. A programmer filed a bug report. To see it, visit <https://bugs.openjdk.java.net/browse/JDK-4252539>. In 1999, that bug was marked as “closed, will not be fixed.” A Sun engineer added an explanation that the Java Virtual Machine Specification does not mandate that `main` is `public` and that “fixing it will cause potential troubles.” Fortunately, sanity prevailed. The Java launcher in Java 1.4 and beyond enforces that the `main` method is `public`.

Of course, it is frustrating to have quality assurance engineers, who are often overworked and not always experts in the fine points of Java, make questionable decisions about bug reports. But it is remarkable that the bug reports and their resolutions have been available for anyone to scrutinize, long before Java was open source.

Notice the braces { } in the source code. In Java, as in C/C++, braces delineate the parts (usually called *blocks*) in your program. In Java, the code for any method must be started by an opening brace { and ended by a closing brace }.

Brace styles have inspired an inordinate amount of useless controversy. This book follows a style that lines up matching braces. As whitespace is irrelevant to the Java compiler, you can use whatever brace style you like.

For now, don't worry about the keywords `static` `void`—just think of them as part of what you need to get a Java program to compile. By the end of Chapter 4, you will understand this incantation completely. The point to remember for now is that every Java application must have a `main` method that is declared in the following way:

```
public class ClassName
{
    public static void main(String[] args)
    {
        program statements
    }
}
```



C++ NOTE: As a C++ programmer, you know what a class is. Java classes are similar to C++ classes, but there are a few differences that can trap you. For example, in Java *all* functions are methods of some class. (The standard terminology refers to them as methods, not member functions.) Thus, in Java you must have a shell class for the `main` method. You may also be familiar with the idea of *static member functions* in C++. These are member functions defined inside a class that do not operate on objects. The `main` method in Java is always static. Finally, as in C/C++, the `void` keyword indicates that this method does not return a value. Unlike C/C++, the `main` method does not return an “exit code” to the operating system. If the `main` method exits normally, the Java program has the exit code 0, indicating successful completion. To terminate the program with a different exit code, use the `System.exit` method.

Next, turn your attention to this fragment:

```
{
    System.out.println("We will not use 'Hello, World!'");
}
```

Braces mark the beginning and end of the *body* of the method. This method has only one statement in it. As with most programming languages, you can think of Java statements as sentences of the language. In Java, every statement

must end with a semicolon. In particular, carriage returns do not mark the end of a statement, so statements can span multiple lines if need be.

The body of the `main` method contains a statement that outputs a single line of text to the console.

Here, we are using the `System.out` object and calling its `println` method. Notice the periods used to invoke a method. Java uses the general syntax

```
object.method(parameters)
```

as its equivalent of a function call.

In this case, the `println` method receives a string parameter. The method displays the string parameter on the console. It then terminates the output line, so that each call to `println` displays its output on a new line. Notice that Java, like C/C++, uses double quotes to delimit strings. (You can find more information about strings later in this chapter.)

Methods in Java, like functions in any programming language, can use zero, one, or more *parameters* (some programmers call them *arguments*). Even if a method takes no parameters, you must still use empty parentheses. For example, a variant of the `println` method with no parameters just prints a blank line. You invoke it with the call

```
System.out.println();
```



NOTE: `System.out` also has a `print` method that doesn't add a newline character to the output. For example, `System.out.print("Hello")` prints `Hello` without a newline. The next output appears immediately after the letter `o`.

3.2 Comments

Comments in Java, as in most programming languages, do not show up in the executable program. Thus, you can add as many comments as needed without fear of bloating the code. Java has three ways of marking comments. The most common form is a `//`. Use this for a comment that runs from the `//` to the end of the line.

```
System.out.println("We will not use 'Hello, World!'); // is this too cute?
```

When longer comments are needed, you can mark each line with a `//`, or you can use the `/*` and `*/` comment delimiters that let you block off a longer comment.

Finally, a third kind of comment is used to generate documentation automatically. This comment uses a `/**` to start and a `*/` to end. You can see this type of comment in Listing 3.1. For more on this type of comment and on automatic documentation generation, see Chapter 4.

Listing 3.1 FirstSample/FirstSample.java

```
1 /**
2  * This is the first sample program in Core Java Chapter 3
3  * @version 1.01 1997-03-22
4  * @author Gary Cornell
5  */
6 public class FirstSample
7 {
8     public static void main(String[] args)
9     {
10         System.out.println("We will not use 'Hello, World!'");
11     }
12 }
```



CAUTION: `/* */` comments do not nest in Java. That is, you might not be able to deactivate code simply by surrounding it with `/*` and `*/` because the code you want to deactivate might itself contain a `*/` delimiter.

3.3 Data Types

Java is a *strongly typed language*. This means that every variable must have a declared type. There are eight *primitive types* in Java. Four of them are integer types; two are floating-point number types; one is the character type `char`, used for code units in the Unicode encoding scheme (see Section 3.3.3, “The `char` Type,” on p. 43); and one is a boolean type for truth values.



NOTE: Java has an arbitrary-precision arithmetic package. However, “big numbers,” as they are called, are Java *objects* and not a primitive Java type. You will see how to use them later in this chapter.

3.3.1 Integer Types

The integer types are for numbers without fractional parts. Negative values are allowed. Java provides the four integer types shown in Table 3.1.

Table 3.1 Java Integer Types

Type	Storage Requirement	Range (Inclusive)
int	4 bytes	−2,147,483,648 to 2,147,483,647 (just over 2 billion)
short	2 bytes	−32,768 to 32,767
long	8 bytes	−9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
byte	1 byte	−128 to 127

In most situations, the `int` type is the most practical. If you want to represent the number of inhabitants of our planet, you'll need to resort to a `long`. The `byte` and `short` types are mainly intended for specialized applications, such as low-level file handling, or for large arrays when storage space is at a premium.

Under Java, the ranges of the integer types do not depend on the machine on which you will be running the Java code. This alleviates a major pain for the programmer who wants to move software from one platform to another, or even between operating systems on the same platform. In contrast, C and C++ programs use the most efficient integer type for each processor. As a result, a C program that runs well on a 32-bit processor may exhibit integer overflow on a 16-bit system. Since Java programs must run with the same results on all machines, the ranges for the various types are fixed.

Long integer numbers have a suffix `L` or `l` (for example, `4000000000L`). Hexadecimal numbers have a prefix `0x` or `0X` (for example, `0xCAFE`). Octal numbers have a prefix `0` (for example, `010` is 8)—naturally, this can be confusing, and few programmers use octal constants.

You can write numbers in binary, with a prefix `0b` or `0B`. For example, `0b1001` is 9. You can add underscores to number literals, such as `1_000_000` (or `0b1111_0100_0010_0100_0000`) to denote one million. The underscores are for human eyes only. The Java compiler simply removes them.



C++ NOTE: In C and C++, the sizes of types such as `int` and `long` depend on the target platform. On a 16-bit processor such as the 8086, integers are 2 bytes, but on a 32-bit processor like a Pentium or SPARC they are 4-byte quantities. Similarly, `long` values are 4-byte on 32-bit processors and 8-byte on 64-bit processors. These differences make it challenging to write cross-platform programs. In Java, the sizes of all numeric types are platform-independent.

Note that Java does not have any unsigned versions of the `int`, `long`, `short`, or `byte` types.



NOTE: If you work with integer values that can never be negative and you really need an additional bit, you can, with some care, interpret signed integer values as unsigned. For example, instead of having a byte value `b` represent the range from -128 to 127 , you may want a range from 0 to 255 . You can store it in a byte. Due to the nature of binary arithmetic, addition, subtraction, and multiplication will work provided they don't overflow. For other operations, call `Byte.toUnsignedInt(b)` to get an `int` value between 0 and 255 , then process the integer value and cast back to `byte`. The `Integer` and `Long` classes have methods for unsigned division and remainder.

3.3.2 Floating-Point Types

The floating-point types denote numbers with fractional parts. The two floating-point types are shown in Table 3.2.

Table 3.2 Floating-Point Types

Type	Storage Requirement	Range
<code>float</code>	4 bytes	Approximately $\pm 3.40282347 \times 10^{38}$ (6–7 significant decimal digits)
<code>double</code>	8 bytes	Approximately $\pm 1.79769313486231570 \times 10^{308}$ (15 significant decimal digits)

The name `double` refers to the fact that these numbers have twice the precision of the `float` type. (Some people call these *double-precision* numbers.) The limited precision of `float` (6–7 significant digits) is simply not sufficient for many situations. Use `float` values only when you work with a library that requires them, or when you need to store a very large number of them.

Numbers of type `float` have a suffix `F` or `f` (for example, `3.14F`). Floating-point numbers without an `F` suffix (such as `3.14`) are always considered to be of type `double`. You can optionally supply the `D` or `d` suffix (for example, `3.14D`).

An `E` or `e` denotes a decimal exponent. For example, `1.729E3` is the same as `1729`.



NOTE: You can specify floating-point literals in hexadecimal. For example, $0.125 = 2^{-3}$ can be written as `0x1.0p-3`. In hexadecimal notation, you use a `p`, not an `e`, to denote the exponent. (An `e` is a hexadecimal digit.) Note that the mantissa is written in hexadecimal and the exponent in decimal. The base of the exponent is 2 , not 10 .

All floating-point computations follow the IEEE 754 specification. In particular, there are three special floating-point values to denote overflows and errors:

- Positive infinity
- Negative infinity
- NaN (not a number)

For example, the result of dividing a positive number by 0 is positive infinity. Computing 0/0 or the square root of a negative number yields NaN.



NOTE: The constants `Double.POSITIVE_INFINITY`, `Double.NEGATIVE_INFINITY`, and `Double.NaN` (as well as corresponding `Float` constants) represent these special values, but they are rarely used in practice. In particular, you cannot test

```
if (x == Double.NaN) // is never true
```

to check whether a particular result equals `Double.NaN`. All “not a number” values are considered distinct. However, you can use the `Double.isNaN` method:

```
if (Double.isNaN(x)) // check whether x is "not a number"
```



CAUTION: Floating-point numbers are *not* suitable for financial calculations in which roundoff errors cannot be tolerated. For example, the command `System.out.println(2.0 - 1.1)` prints `0.8999999999999999`, not `0.9` as you would expect. Such roundoff errors are caused by the fact that floating-point numbers are represented in the binary number system. There is no precise binary representation of the fraction `1/10`, just as there is no accurate representation of the fraction `1/3` in the decimal system. If you need precise numerical computations without roundoff errors, use the `BigDecimal` class, which is introduced later in this chapter.

3.3.3 The char Type

The `char` type was originally intended to describe individual characters. However, this is no longer the case. Nowadays, some Unicode characters can be described with one `char` value, and other Unicode characters require two `char` values. Read the next section for the gory details.

Literal values of type `char` are enclosed in single quotes. For example, `'A'` is a character constant with value 65. It is different from `"A"`, a string containing a single character. Values of type `char` can be expressed as hexadecimal values that run from `\u0000` to `\uFFFF`. For example, `\u2122` is the trademark symbol (TM) and `\u03C0` is the Greek letter pi (π).

Besides the `\u` escape sequences, there are several escape sequences for special characters, as shown in Table 3.3. You can use these escape sequences inside quoted character literals and strings, such as `'\u2122'` or `"Hello\n"`. The `\u` escape sequence (but none of the other escape sequences) can even be used *outside* quoted character constants and strings. For example,

```
public static void main(String\u005B\u005D args)
```

is perfectly legal—`\u005B` and `\u005D` are the encodings for `[` and `]`.

Table 3.3 Escape Sequences for Special Characters

Escape Sequence	Name	Unicode Value
<code>\b</code>	Backspace	<code>\u0008</code>
<code>\t</code>	Tab	<code>\u0009</code>
<code>\n</code>	Line feed	<code>\u000a</code>
<code>\r</code>	Carriage return	<code>\u000d</code>
<code>\f</code>	Form feed	<code>\u000c</code>
<code>\"</code>	Double quote	<code>\u0022</code>
<code>\'</code>	Single quote	<code>\u0027</code>
<code>\\</code>	Backslash	<code>\u005c</code>
<code>\s</code>	Space. Used in text blocks to retain trailing whitespace.	<code>\u0020</code>
<code>\newline</code>	In text blocks only: Join this line with the next	—



CAUTION: Unicode escape sequences are processed before the code is parsed. For example, `"\u0022+\u0022"` is *not* a string consisting of a plus sign surrounded by quotation marks (U+0022). Instead, the `\u0022` are converted into `"` before parsing, yielding `"\""`, or an empty string.

Even more insidiously, you must beware of `\u` inside comments. The comment

```
// \u000A is a newline
```

yields a syntax error since `\u000A` is replaced with a newline when the program is read. Similarly, a comment

```
// look inside c:\users
```

yields a syntax error because the `\u` is not followed by four hex digits.



NOTE: You can have any number of `u` in a Unicode escape sequence: `\uu0022` and `\uuu0022` also denote the quotation mark character. There is a reason for this oddity. Consider a programmer happily coding in Unicode who is forced, for some archaic reason, to check in code as ASCII only. A conversion tool can turn any character `> U+007F` into a Unicode escape and add a `u` to every existing Unicode escape. That makes the conversion reversible. For example, `\ud800`™ is turned into `\uud800 \u2122` and can be converted back to `\ud800`™.

3.3.4 Unicode and the char Type

To fully understand the `char` type, you have to know about the Unicode encoding scheme. Unicode was invented to overcome the limitations of traditional character encoding schemes. Before Unicode, there were many different standards: ASCII in the United States, ISO 8859-1 for Western European languages, KOI-8 for Russian, GB18030 and BIG-5 for Chinese, and so on. This caused two problems. First, a particular code value corresponds to different letters in the different encoding schemes. Second, the encodings for languages with large character sets have variable length: Some common characters are encoded as single bytes, others require two or more bytes.

Unicode was designed to solve these problems. When the unification effort started in the 1980s, a fixed 2-byte code was more than sufficient to encode all characters used in all languages in the world, with room to spare for future expansion—or so everyone thought at the time. In 1991, Unicode 1.0 was released, using slightly less than half of the available 65,536 code values. Java was designed from the ground up to use 16-bit Unicode characters, which was a major advance over other programming languages that used 8-bit characters.

Unfortunately, over time, the inevitable happened. Unicode grew beyond 65,536 characters, primarily due to the addition of a very large set of ideographs used for Chinese, Japanese, and Korean. Now, the 16-bit `char` type is insufficient to describe all Unicode characters.

We need a bit of terminology to explain how this problem is resolved in Java. A *code point* is a code value that is associated with a character in an encoding scheme. In the Unicode standard, code points are written in hexadecimal and prefixed with `U+`, such as `U+0041` for the code point of the Latin letter A. Unicode has code points that are grouped into 17 *code planes*. The first code plane, called the *basic multilingual plane*, consists of the “classic” Unicode characters with code points `U+0000` to `U+FFFF`. Sixteen additional planes, with code points `U+10000` to `U+10FFFF`, hold the *supplementary characters*.

The UTF-16 encoding represents all Unicode code points in a variable-length code. The characters in the basic multilingual plane are represented as 16-bit values, called *code units*. The supplementary characters are encoded as consecutive pairs of code units. Each of the values in such an encoding pair falls into a range of 2048 unused values of the basic multilingual plane, called the *surrogates area* (U+D800 to U+DBFF for the first code unit, U+DC00 to U+DFFF for the second code unit). This is rather clever, because you can immediately tell whether a code unit encodes a single character or it is the first or second part of a supplementary character. For example, ☉ (the mathematical symbol for the set of octonions, <http://math.ucr.edu/home/baez/octonions>) has code point U+1D546 and is encoded by the two code units U+D835 and U+DD46. (See <https://tools.ietf.org/html/rfc2781> for a description of the encoding algorithm.)

In Java, the `char` type describes a *code unit* in the UTF-16 encoding.

I strongly recommend against using the `char` type in your programs unless you are actually manipulating UTF-16 code units. You are almost always better off treating strings as abstract data types—see Section 3.6, “Strings,” on p. 61.

3.3.5 The `boolean` Type

The `boolean` type has two values, `false` and `true`. It is used for evaluating logical conditions. You cannot convert between integers and `boolean` values.



C++ NOTE: In C++, numbers and even pointers can be used in place of `boolean` values. The value `0` is equivalent to the `bool` value `false`, and a nonzero value is equivalent to `true`. This is *not* the case in Java. Thus, Java programmers are shielded from accidents such as

```
if (x = 0) // oops... meant x == 0
```

In C++, this test compiles and runs, always evaluating to `false`. In Java, the test does not compile because the integer expression `x = 0` cannot be converted to a `boolean` value.

3.4 Variables and Constants

As in every programming language, variables are used to store values. Constants are variables whose values don't change. In the following sections, you will learn how to declare variables and constants.

3.4.1 Declaring Variables

In Java, every variable has a *type*. You declare a variable by placing the type first, followed by the name of the variable. Here are some examples:

```
double salary;  
int vacationDays;  
long earthPopulation;  
boolean done;
```

Notice the semicolon at the end of each declaration. The semicolon is necessary because a declaration is a complete Java statement, which must end in a semicolon.

The identifier for a variable name (as well as for other names) is made up of letters, digits, currency symbols, and “punctuation connectors.” The first character cannot be a digit.

Symbols like '+' or '@' cannot be used inside variable names, nor can spaces. Letter case is significant: `main` and `Main` are distinct identifiers. The length of an identifier is essentially unlimited.

The terms “letter,” “digit,” and “currency symbol” are much broader in Java than in most languages. A letter is *any* Unicode character that denotes a letter in a language. For example, German users can use umlauts such as 'ä' in variable names; Greek speakers could use a π . Similarly, digits are 0–9 and *any* Unicode characters that denote a digit. Currency symbols are \$, €, ¥, and so on. Punctuation connectors include the underscore character `_`, a “wavy low line” `~`, and a few others. In practice, most programmers stick to A–Z, a–z, 0–9, and the underscore `_`.



TIP: If you are really curious as to what Unicode characters can be used in identifiers, you can use the `isJavaIdentifierStart` and `isJavaIdentifierPart` methods in the `Character` class to check.



TIP: Even though \$ is a valid character in an identifier, you should not use it in your own code. It is intended for names that are generated by the Java compiler and other tools.

You also cannot use a Java keyword such as `class` as a variable name.

As of Java 9, a single underscore `_` is a reserved word. A future version of Java may use `_` as a wildcard symbol.

You can declare multiple variables on a single line:

```
int i, j; // both are integers
```

However, I don't recommend this style. If you declare each variable separately, your programs are easier to read.



NOTE: As you saw, names are case sensitive, for example, `hireday` and `hireDay` are two separate names. In general, you should not have two names that only differ in their letter case. However, sometimes it is difficult to come up with a good name for a variable. Many programmers then give the variable the same name as the type, for example

```
Box box; // "Box" is the type and "box" is the variable name
```

Other programmers prefer to use an "a" prefix for the variable:

```
Box aBox;
```

3.4.2 Initializing Variables

After you declare a variable, you must explicitly initialize it by means of an assignment statement—you can never use the value of an uninitialized variable. For example, the Java compiler flags the following sequence of statements as an error:

```
int vacationDays;  
System.out.println(vacationDays); // ERROR--variable not initialized
```

You assign to a previously declared variable by using the variable name on the left, an equal sign (=), and then some Java expression with an appropriate value on the right.

```
int vacationDays;  
vacationDays = 12;
```

You can both declare and initialize a variable on the same line. For example:

```
int vacationDays = 12;
```

Finally, in Java you can put declarations anywhere in your code. For example, the following is valid code in Java:

```
double salary = 65000.0;  
System.out.println(salary);  
int vacationDays = 12; // OK to declare a variable here
```

In Java, it is considered good style to declare variables as closely as possible to the point where they are first used.



NOTE: Starting with Java 10, you do not need to declare the types of local variables if they can be inferred from the initial value. Simply use the keyword `var` instead of the type:

```
var vacationDays = 12; // vacationDays is an int
var greeting = "Hello"; // greeting is a String
```

As you will see in the next chapter, this feature can make the declaration of objects less verbose.



C++ NOTE: C and C++ distinguish between the *declaration* and *definition* of a variable. For example,

```
int i = 10;
```

is a definition, whereas

```
extern int i;
```

is a declaration. In Java, no declarations are separate from definitions.

3.4.3 Constants

In Java, you use the keyword `final` to denote a constant. For example:

```
public class Constants
{
    public static void main(String[] args)
    {
        final double CM_PER_INCH = 2.54;
        double paperWidth = 8.5;
        double paperHeight = 11;
        System.out.println("Paper size in centimeters: "
            + paperWidth * CM_PER_INCH + " by " + paperHeight * CM_PER_INCH);
    }
}
```

The keyword `final` indicates that you can assign to the variable once, and then its value is set once and for all. It is customary to name constants in all uppercase.

It is probably more common in Java to create a constant so it's available to multiple methods inside a single class. These are usually called *class constants*.

Set up a class constant with the keywords `static final`. Here is an example of using a class constant:

```
public class Constants2
{
    public static final double CM_PER_INCH = 2.54;

    public static void main(String[] args)
    {
        double paperWidth = 8.5;
        double paperHeight = 11;
        System.out.println("Paper size in centimeters: "
            + paperWidth * CM_PER_INCH + " by " + paperHeight * CM_PER_INCH);
    }
}
```

Note that the definition of the class constant appears *outside* the `main` method. Thus, the constant can also be used in other methods of the same class. Furthermore, if the constant is declared, as in this example, `public`, methods of other classes can also use it—in our example, as `Constants2.CM_PER_INCH`.



C++ NOTE: `const` is a reserved Java keyword, but it is not currently used for anything. You must use `final` for a constant.

3.4.4 Enumerated Types

Sometimes, a variable should only hold a restricted set of values. For example, you may sell clothes or pizza in four sizes: small, medium, large, and extra large. Of course, you could encode these sizes as integers 1, 2, 3, 4 or characters S, M, L, and X. But that is an error-prone setup. It is too easy for a variable to hold a wrong value (such as 0 or m).

You can define your own *enumerated type* whenever such a situation arises. An enumerated type has a finite number of named values. For example,

```
enum Size { SMALL, MEDIUM, LARGE, EXTRA_LARGE };
```

Now you can declare variables of this type:

```
Size s = Size.MEDIUM;
```

A variable of type `Size` can hold only one of the values listed in the type declaration, or the special value `null` that indicates that the variable is not set to any value at all.

Enumerated types are discussed in greater detail in Chapter 5.

3.5 Operators

Operators are used to combine values. As you will see in the following sections, Java has a rich set of arithmetic and logical operators and mathematical functions.

3.5.1 Arithmetic Operators

The usual arithmetic operators `+`, `-`, `*`, `/` are used in Java for addition, subtraction, multiplication, and division. The `/` operator denotes integer division if both arguments are integers, and floating-point division otherwise. Integer remainder (sometimes called *modulus*) is denoted by `%`. For example, `15 / 2` is 7, `15 % 2` is 1, and `15.0 / 2` is 7.5.

Note that integer division by 0 raises an exception, whereas floating-point division by 0 yields an infinite or NaN result.



NOTE: One of the stated goals of the Java programming language is portability. A computation should yield the same results no matter which virtual machine executes it. For arithmetic computations with floating-point numbers, it can be surprisingly difficult to achieve this portability. The `double` type uses 64 bits to store a numeric value, but some processors use 80-bit floating-point registers. These registers yield added precision in intermediate steps of a computation.

But the result may be *different* from a computation that uses 64 bits throughout. For perfect portability, the initial specification of the Java virtual machine mandated that all intermediate computations must be truncated to 64 bit. The numeric community hated it. The computations were *slower* because the truncation operations took time on popular processors. For that reason, the Java programming language was updated to recognize the conflicting demands for optimum performance and perfect reproducibility. Virtual machine designers were permitted to use extended precision for intermediate computations. However, methods tagged with the `strictfp` keyword had to use strict floating-point operations that yield reproducible results.

Processors have become more flexible, and they can now carry out 64-bit arithmetic efficiently. As of Java 17, the virtual machine is again required to carry out strict 64-bit arithmetic, and the `strictfp` keyword is now obsolete.

3.5.2 Mathematical Functions and Constants

The `Math` class contains an assortment of mathematical functions that you may occasionally need, depending on the kind of programming that you do.

To take the square root of a number, use the `sqrt` method:

```
double x = 4;  
double y = Math.sqrt(x);  
System.out.println(y); // prints 2.0
```



NOTE: There is a subtle difference between the `println` method and the `sqrt` method. The `println` method operates on the `System.out` object. But the `sqrt` method in the `Math` class does not operate on any object. Such a method is called a *static* method. You can learn more about static methods in Chapter 4.

The Java programming language has no operator for raising a quantity to a power: You must use the `pow` method in the `Math` class. The statement

```
double y = Math.pow(x, a);
```

sets `y` to be `x` raised to the power `a` (x^a). The `pow` method's parameters are both of type `double`, and it returns a `double` as well.

The `floorMod` method aims to solve a long-standing problem with integer remainders. Consider the expression `n % 2`. Everyone knows that this is 0 if `n` is even and 1 if `n` is odd. Except, of course, when `n` is odd and negative. Then it is -1. Why? When the first computers were built, someone had to make rules for how integer division and remainder should work for negative operands. Mathematicians had known the optimal (or "Euclidean") rule for a few hundred years: always leave the remainder ≥ 0 . But, rather than open a math textbook, those pioneers came up with rules that seemed reasonable but are actually inconvenient.

Consider this problem. You compute the position of the hour hand of a clock. An adjustment is applied, and you want to normalize to a number between 0 and 11. That is easy: `(position + adjustment) % 12`. But what if the adjustment is negative? Then you might get a negative number. So you have to introduce a branch, or use `((position + adjustment) % 12 + 12) % 12`. Either way, it is a hassle.

The `floorMod` method makes it easier: `floorMod(position + adjustment, 12)` always yields a value between 0 and 11. (Unfortunately, `floorMod` gives negative results for negative divisors, but that situation doesn't often occur in practice.)

The `Math` class supplies the usual trigonometric functions:

```
Math.sin  
Math.cos  
Math.tan  
Math.atan  
Math.atan2
```

and the exponential function with its inverse, the natural logarithm, as well as the decimal logarithm:

```
Math.exp  
Math.log  
Math.log10
```

Finally, two constants denote the closest possible approximations to the mathematical constants π and e :

```
Math.PI  
Math.E
```



TIP: You can avoid the `Math` prefix for the mathematical methods and constants by adding the following line to the top of your source file:

```
import static java.lang.Math.*;
```

For example:

```
System.out.println("The square root of \u03C0 is " + sqrt(PI));
```

Static imports are covered in Chapter 4.



NOTE: The methods in the `Math` class use the routines in the computer's floating-point unit for fastest performance. If completely predictable results are more important than performance, use the `StrictMath` class instead. It implements the algorithms from the “Freely Distributable Math Library” (www.netlib.org/fdlibm), guaranteeing identical results on all platforms.



NOTE: The `Math` class provides several methods to make integer arithmetic safer. The mathematical operators quietly return wrong results when a computation overflows. For example, one billion times three (`1000000000 * 3`) evaluates to `-1294967296` because the largest `int` value is just over two billion. If you call `Math.multiplyExact(1000000000, 3)` instead, an exception is generated. You can catch that exception or let the program terminate rather than quietly continue with a wrong result. There are also methods `addExact`, `subtractExact`, `incrementExact`, `decrementExact`, `negateExact`, and `absExact`, all with `int` and `long` parameters.

3.5.3 Conversions between Numeric Types

It is often necessary to convert from one numeric type to another. Figure 3.1 shows the legal conversions.

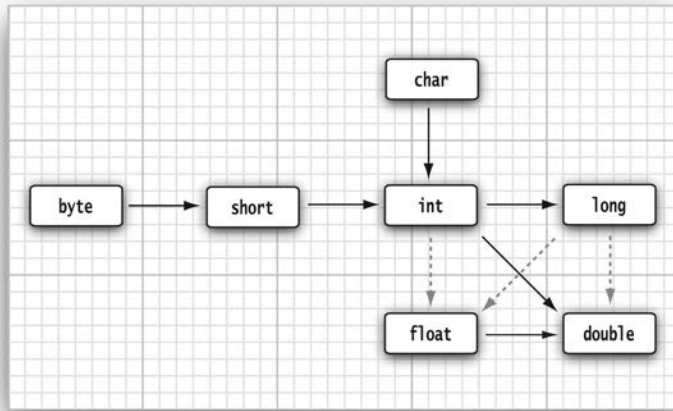


Figure 3.1 Legal conversions between numeric types

The six solid arrows in Figure 3.1 denote conversions without information loss. The three dotted arrows denote conversions that may lose precision. For example, a large integer such as 123456789 has more digits than the float type can represent. When the integer is converted to a float, the resulting value has the correct magnitude but loses some precision.

```
int n = 123456789;
float f = n; // f is 1.23456792E8
```

When two values are combined with a binary operator (such as `n + f` where `n` is an integer and `f` is a floating-point value), both operands are converted to a common type before the operation is carried out.

- If either of the operands is of type `double`, the other one will be converted to a `double`.
- Otherwise, if either of the operands is of type `float`, the other one will be converted to a `float`.
- Otherwise, if either of the operands is of type `long`, the other one will be converted to a `long`.
- Otherwise, both operands will be converted to an `int`.

3.5.4 Casts

In the preceding section, you saw that `int` values are automatically converted to `double` values when necessary. On the other hand, there are obviously times

when you want to consider a `double` as an integer. Numeric conversions are possible in Java, but of course information may be lost. Conversions in which loss of information is possible are done by means of *casts*. The syntax for casting is to give the target type in parentheses, followed by the variable name. For example:

```
double x = 9.997;  
int nx = (int) x;
```

Now, the variable `nx` has the value 9 because casting a floating-point value to an integer discards the fractional part.

If you want to *round* a floating-point number to the *nearest* integer (which in most cases is a more useful operation), use the `Math.round` method:

```
double x = 9.997;  
int nx = (int) Math.round(x);
```

Now the variable `nx` has the value 10. You still need to use the cast `(int)` when you call `round`. The reason is that the return value of the `round` method is a `long`, and a `long` can only be assigned to an `int` with an explicit cast because there is the possibility of information loss.



CAUTION: If you try to cast a number of one type to another that is out of range for the target type, the result will be a truncated number that has a different value. For example, (byte) 300 is actually 44.



C++ NOTE: You cannot cast between `boolean` values and any numeric type. This convention prevents common errors. In the rare case when you want to convert a `boolean` value to a number, you can use a conditional expression such as `b ? 1 : 0`.

3.5.5 Assignment

There is a convenient shortcut for using binary operators in an assignment. For example,

```
x += 4;
```

is equivalent to

```
x = x + 4;
```

(In general, place the operator to the left of the `=` sign, such as `*=` or `%=`.)



CAUTION: If the operator yields a value whose type is different from that of the left-hand side, then it is coerced to fit. For example, if `x` is an `int`, then the statement

```
x += 3.5;
```

is valid, setting `x` to `(int)(x + 3.5)`.

Note that in Java, an assignment is an *expression*. That is, it has a value—namely, the value that is being assigned. You can use that value—for example, to assign it to another variable. Consider these statements:

```
int x = 1;
int y = x += 4;
```

The value of `x += 4` is 5, since that's the value that is being assigned to `x`. Next, that value is assigned to `y`.

Many programmers find such nested assignments confusing and prefer to write them more clearly, like this:

```
int x = 1;
x += 4;
int y = x;
```

3.5.6 Increment and Decrement Operators

Programmers, of course, know that one of the most common operations with a numeric variable is to add or subtract 1. Java, following in the footsteps of C and C++, has both increment and decrement operators: `n++` adds 1 to the current value of the variable `n`, and `n--` subtracts 1 from it. For example, the code

```
int n = 12;
n++;
```

changes `n` to 13. Since these operators change the value of a variable, they cannot be applied to numbers themselves. For example, `4++` is not a legal statement.

There are two forms of these operators; you've just seen the postfix form of the operator that is placed after the operand. There is also a prefix form, `++n`. Both change the value of the variable by 1. The difference between the two appears only when they are used inside expressions. The prefix form does the addition first; the postfix form evaluates to the old value of the variable.

```
int m = 7;
int n = 7;
int a = 2 * ++m; // now a is 16, m is 8
int b = 2 * n++; // now b is 14, n is 8
```

Many programmers find this behavior confusing. In Java, using ++ inside expressions is uncommon.

3.5.7 Relational and boolean Operators

Java has the full complement of relational operators. To test for equality, use a double equal sign, ==. For example, the value of

```
3 == 7
```

is false.

Use a != for inequality. For example, the value of

```
3 != 7
```

is true.

Finally, you have the usual < (less than), > (greater than), <= (less than or equal), and >= (greater than or equal) operators.

Java, following C++, uses && for the logical “and” operator and || for the logical “or” operator. As you can easily remember from the != operator, the exclamation point ! is the logical negation operator. The && and || operators are evaluated in “short circuit” fashion: The second argument is not evaluated if the first argument already determines the value. If you combine two expressions with the && operator,

```
expression1 && expression2
```

and the truth value of the first expression has been determined to be false, then it is impossible for the result to be true. Thus, the value for the second expression is *not* calculated. This behavior can be exploited to avoid errors. For example, in the expression

```
x != 0 && 1 / x > x + y // no division by 0
```

the second part is never evaluated if x equals zero. Thus, 1 / x is not computed if x is zero, and no divide-by-zero error can occur.

Similarly, the value of *expression*₁ || *expression*₂ is automatically true if the first expression is true, without evaluating the second expression.

3.5.8 The Conditional Operator

Java provides the *conditional* `?:` operator that selects a value, depending on a Boolean expression. The expression

condition ? expression₁ : expression₂

evaluates to the first expression if the condition is true, to the second expression otherwise. For example,

x < y ? x : y

gives the smaller of *x* and *y*.

3.5.9 Switch Expressions

If you need to choose among more than two values, then you can use a `switch` expression (as of Java 14). It looks like this:

```
String seasonName = switch (seasonCode)
{
    case 0 -> "Spring";
    case 1 -> "Summer";
    case 2 -> "Fall";
    case 3 -> "Winter";
    default -> "???";
};
```

The case labels can also be strings or constants of an enumerated type.



NOTE: The `switch` expression, like every expression, has a value. Note the `->` arrow preceding the value in each branch.



NOTE: As of Java 14, there are four (!) forms of `switch`. This section focuses on the most useful one. See Section 3.8.5, “Multiple Selections with `switch`,” on p. 98 for a thorough discussion of all forms of `switch` expressions and statements.

You can provide multiple labels for each case, separated by commas:

```
int numLetters = switch (seasonName)
{
    case "Spring", "Summer", "Winter" -> 6;
    case "Fall" -> 4;
    default -> -1;
};
```

When you use the `switch` expression with enumerated constants, you need not supply the name of the enumeration in each label—it is deduced from the `switch` value. For example:

```
enum Size { SMALL, MEDIUM, LARGE, EXTRA_LARGE };
...
Size itemSize = ...;
String label = switch (itemSize)
{
    case SMALL -> "S"; // no need to use Size.SMALL
    case MEDIUM -> "M";
    case LARGE -> "L";
    case EXTRA_LARGE -> "XL";
};
```

In the example, it was legal to omit the default since there was a case for each possible value.



CAUTION: A `switch` expression with an integer or `String` operand must always have a default since it must yield a value, no matter what the operand value is.



CAUTION: If the operand is `null`, a `NullPointerException` is thrown.

3.5.10 Bitwise Operators

For any of the integer types, you have operators that can work directly with the bits that make up the integers. This means that you can use masking techniques to get at individual bits in a number. The bitwise operators are

`&` (“and”) `|` (“or”) `^` (“xor”) `~` (“not”)

These operators work on bit patterns. For example, if `n` is an integer variable, then

```
int fourthBitFromRight = (n & 0b1000) / 0b1000;
```

gives you a 1 if the fourth bit from the right in the binary representation of `n` is 1, and 0 otherwise. Using `&` with the appropriate power of 2 lets you mask out all but a single bit.



NOTE: When applied to `boolean` values, the `&` and `|` operators yield a `boolean` value. These operators are similar to the `&&` and `||` operators, except that the `&` and `|` operators are not evaluated in “short circuit” fashion—that is, both arguments are evaluated before the result is computed.

There are also `>>` and `<<` operators which shift a bit pattern right or left. These operators are convenient when you need to build up bit patterns to do bit masking:

```
int fourthBitFromRight = (n & (1 << 3)) >> 3;
```

Finally, a `>>>` operator fills the top bits with zero, unlike `>>` which extends the sign bit into the top bits. There is no `<<<` operator.



CAUTION: The right-hand argument of the shift operators is reduced modulo 32 (unless the left-hand argument is a `long`, in which case the right-hand argument is reduced modulo 64). For example, the value of `1 << 35` is the same as `1 << 3` or 8.



C++ NOTE: In C/C++, there is no guarantee as to whether `>>` performs an arithmetic shift (extending the sign bit) or a logical shift (filling in with zeroes). Implementors are free to choose whichever is more efficient. That means the C/C++ `>>` operator may yield implementation-dependent results for negative numbers. Java removes that uncertainty.

3.5.11 Parentheses and Operator Hierarchy

Table 3.4 shows the precedence of operators. If no parentheses are used, operations are performed in the hierarchical order indicated. Operators on the same level are processed from left to right, except for those that are right-associative, as indicated in the table. For example, `&&` has a higher precedence than `||`, so the expression

```
a && b || c
```

means

```
(a && b) || c
```

Since `+=` associates right to left, the expression

```
a += b += c
```

means

```
a += (b += c)
```

That is, the value of `b += c` (which is the value of `b` after the addition) is added to `a`.

Table 3.4 Operator Precedence

Operators	Associativity
<code>[] . ()</code> (method call)	Left to right
<code>! ~ ++ -- + (unary) - (unary) () (cast) new</code>	Right to left
<code>* / %</code>	Left to right
<code>+ -</code>	Left to right
<code><< >> >>></code>	Left to right
<code>< <= > >= instanceof</code>	Left to right
<code>== !=</code>	Left to right
<code>&</code>	Left to right
<code>^</code>	Left to right
<code> </code>	Left to right
<code>&&</code>	Left to right
<code> </code>	Left to right
<code>?:</code>	Right to left
<code>= += -= *= /= %= &= = ^= <<= >>= >>>=</code>	Right to left



C++ NOTE: Unlike C or C++, Java does not have a comma operator. However, you can use a *comma-separated list of expressions* in the first and third slot of a `for` statement.

3.6 Strings

Conceptually, Java strings are sequences of Unicode characters. For example, the string `"Java\u2122"` consists of the five Unicode characters `J`, `a`, `v`, `a`, and `™`. Java does not have a built-in string type. Instead, the standard Java library contains a predefined class called, naturally enough, `String`. Each quoted string is an instance of the `String` class:

```
String e = ""; // an empty string
String greeting = "Hello";
```

3.6.1 Substrings

You can extract a substring from a larger string with the `substring` method of the `String` class. For example,

```
String greeting = "Hello";
String s = greeting.substring(0, 3);
```

creates a string consisting of the characters "Hel".



NOTE: Like C and C++, Java counts code units and code points in strings starting with 0.

The second parameter of `substring` is the first position that you *do not* want to copy. In our case, we want to copy positions 0, 1, and 2 (from position 0 to position 2 inclusive). As `substring` counts it, this means from position 0 inclusive to position 3 *exclusive*.

There is one advantage to the way `substring` works: Computing the length of the substring is easy. The string `s.substring(a, b)` always has length $b - a$. For example, the substring "Hel" has length $3 - 0 = 3$.

3.6.2 Concatenation

Java, like most programming languages, allows you to use `+` to join (concatenate) two strings.

```
String expletive = "Expletive";
String PG13 = "deleted";
String message = expletive + PG13;
```

The preceding code sets the variable `message` to the string "Expletivedeleted". (Note the lack of a space between the words: The `+` operator joins two strings in the order received, *exactly* as they are given.)

When you concatenate a string with a value that is not a string, the latter is converted to a string. (As you will see in Chapter 5, every Java object can be converted to a string.) For example,

```
int age = 13;
String rating = "PG" + age;
```

sets `rating` to the string "PG13".

This feature is commonly used in output statements. For example,

```
System.out.println("The answer is " + answer);
```

is perfectly acceptable and prints what you would expect (and with correct spacing because of the space after the word `is`).

If you need to put multiple strings together, separated by a delimiter, use the static `join` method:

```
String all = String.join(" / ", "S", "M", "L", "XL");  
// all is the string "S / M / L / XL"
```

As of Java 11, there is a `repeat` method:

```
String repeated = "Java".repeat(3); // repeated is "JavaJavaJava"
```

3.6.3 Strings Are Immutable

The `String` class gives no methods that let you *change* a character in an existing string. If you want to turn `greeting` into `"Help!"`, you cannot directly change the last positions of `greeting` into `'p'` and `'!'`. If you are a C programmer, this can make you feel pretty helpless. How are we going to modify the string? In Java, it is quite easy: Concatenate the substring that you want to keep with the characters that you want to replace.

```
String greeting = "Hello";  
greeting = greeting.substring(0, 3) + "p!";
```

This declaration changes the current value of the `greeting` variable to `"Help!"`.

Since you cannot change the individual characters in a Java string, the documentation refers to the objects of the `String` class as *immutable*. Just as the number 3 is always 3, the string `"Hello"` will always contain the code-unit sequence for the characters `H`, `e`, `l`, `l`, `o`. You cannot change these values. Yet you can, as you just saw, change the contents of the string *variable* `greeting` and make it refer to a different string, just as you can make a numeric variable currently holding the value 3 hold the value 4.

Isn't that a lot less efficient? It would seem simpler to change the code units than to build up a whole new string from scratch. Well, yes and no. Indeed, it isn't efficient to generate a new string that holds the concatenation of `"Hel"` and `"p!"`. But immutable strings have one great advantage: The compiler can arrange that strings are *shared*.

To understand how this works, think of the various strings as sitting in a common pool. String variables then point to locations in the pool. If you copy a string variable, both the original and the copy share the same characters.

Overall, the designers of Java decided that the efficiency of sharing outweighs the inefficiency of string editing by extracting substrings and concatenating. Look at your own programs; most of the time, you probably don't change strings—you just compare them. (There is one common exception—assembling strings from individual characters or from shorter strings that come from the keyboard or a file. For these situations, Java provides a separate class—see Section 3.6.9, “Building Strings,” on p. 73.)



C++ NOTE: C programmers are generally bewildered when they see Java strings for the first time because they think of strings as arrays of characters:

```
char greeting[] = "Hello";
```

That is a wrong analogy: A Java string is roughly analogous to a `char*` pointer,

```
char* greeting = "Hello";
```

When you replace `greeting` with another string, the Java code does roughly the following:

```
char* temp = malloc(6);
strncpy(temp, greeting, 3);
strncpy(temp + 3, "p!", 3);
greeting = temp;
```

Sure, now `greeting` points to the string "Help!". And even the most hardened C programmer must admit that the Java syntax is more pleasant than a sequence of `strncpy` calls. But what if we make another assignment to `greeting`?

```
greeting = "Howdy";
```

Don't we have a memory leak? After all, the original string was allocated on the heap. Fortunately, Java does automatic garbage collection. If a block of memory is no longer needed, it will eventually be recycled.

If you are a C++ programmer and use the `string` class defined by ANSI C++, you will be much more comfortable with the Java `String` type. C++ `string` objects also perform automatic allocation and deallocation of memory. The memory management is performed explicitly by constructors, assignment operators, and destructors. However, C++ strings are mutable—you can modify individual characters in a string.

3.6.4 Testing Strings for Equality

To test whether two strings are equal, use the `equals` method. The expression

```
s.equals(t)
```

returns true if the strings *s* and *t* are equal, false otherwise. Note that *s* and *t* can be string variables or string literals. For example, the expression

```
"Hello".equals(greeting)
```

is perfectly legal. To test whether two strings are identical except for the upper/lowercase letter distinction, use the `equalsIgnoreCase` method.

```
"Hello".equalsIgnoreCase("hello")
```

Do *not* use the `==` operator to test whether two strings are equal! It only determines whether or not the strings are stored in the same location. Sure, if strings are in the same location, they must be equal. But it is entirely possible to store multiple copies of identical strings in different places.

```
String greeting = "Hello"; // initialize greeting to a string
if (greeting == "Hello") . . .
    // probably true
if (greeting.substring(0, 3) == "Hel") . . .
    // probably false
```

If the virtual machine always arranges for equal strings to be shared, then you could use the `==` operator for testing equality. But only string *literals* are shared, not strings that are the result of operations like `+` or `substring`. Therefore, *never* use `==` to compare strings lest you end up with a program with the worst kind of bug—an intermittent one that seems to occur randomly.



C++ NOTE: If you are used to the C++ string class, you have to be particularly careful about equality testing. The C++ string class does overload the `==` operator to test for equality of the string contents. It is perhaps unfortunate that Java goes out of its way to give strings the same “look and feel” as numeric values but then makes strings behave like pointers for equality testing. The language designers could have redefined `==` for strings, just as they made a special arrangement for `+`. Oh well, every language has its share of inconsistencies.

C programmers never use `==` to compare strings but use `strcmp` instead. The Java method `compareTo` is the exact analog of `strcmp`. You can use

```
if (greeting.compareTo("Hello") == 0) . . .
```

but it seems clearer to use `equals` instead.

3.6.5 Empty and Null Strings

The empty string `""` is a string of length 0. You can test whether a string is empty by calling

```
if (str.length() == 0)
```

or

```
if (str.equals(""))
```

An empty string is a Java object which holds the string length (namely, 0) and an empty contents. However, a `String` variable can also hold a special value, called `null`, that indicates that no object is currently associated with the variable. (See Chapter 4 for more information about `null`.) To test whether a string is `null`, use

```
if (str == null)
```

Sometimes, you need to test that a string is neither `null` nor empty. Then use

```
if (str != null && str.length() != 0)
```

You need to test that `str` is not `null` first. As you will see in Chapter 4, it is an error to invoke a method on a `null` value.

3.6.6 Code Points and Code Units

Java strings are sequences of `char` values. As you saw in Section 3.3.3, “The `char` Type,” on p. 43, the `char` data type is a code unit for representing Unicode code points in the UTF-16 encoding. The most commonly used Unicode characters can be represented with a single code unit. The supplementary characters require a pair of code units.

The `length` method yields the number of code units required for a given string in the UTF-16 encoding. For example:

```
String greeting = "Hello";
int n = greeting.length(); // is 5
```

To get the true length—that is, the number of code points—call

```
int cpCount = greeting.codePointCount(0, greeting.length());
```

The call `s.charAt(n)` returns the code unit at position `n`, where `n` is between 0 and `s.length() - 1`. For example:

```
char first = greeting.charAt(0); // first is 'H'
char last = greeting.charAt(4); // last is 'o'
```

To get at the *i*th code point, use the statements

```
int index = greeting.offsetByCodePoints(0, i);
int cp = greeting.codePointAt(index);
```

What is the fuss about code units? Consider the sentence

① is the set of octonions.